

MOUHIEDDINE SABIR

BSc in Computer Engineering

**EXPLORING SOLUTIONS TO THE TRAVELLING
SALESMAN PROBLEM:
FROM EXACT ALGORITHMS TO THE
RED-BLACK ANT COLONY SYSTEM**

BACHELOR IN COMPUTER ENGINEERING

University of Padua
Settembre, 2024



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

DEPARTMENT OF
INFORMATION ENGINEERING

EXPLORING SOLUTIONS TO THE TRAVELLING SALESMAN PROBLEM: FROM EXACT ALGORITHMS TO THE RED-BLACK ANT COLONY SYSTEM

MOUHIEDDINE SABIR

BSc in Computer Engineering

Adviser: Domenico Salvagnin

Professore Associato, Università degli Studi di Padova

Examination Committee

Chair: Luca Tonin

Professore Associato, Università degli Studi di Padova

Adviser: Domenico Salvagnin

Professore Associato, Università degli Studi di Padova

Anno Accademico: 2023-2024

Data di laurea: 26/09/2024

BACHELOR IN COMPUTER ENGINEERING

University of Padua

Settembre, 2024

I dedicate this thesis to my parents, whose unconditional love, constant support, and infinite sacrifices have made this achievement possible. Your faith in me and your encouragement have been the driving force of this journey. Thank you for being my pillar and my source of inspiration

ACKNOWLEDGEMENTS

I would first like to express my sincere gratitude to my advisor, Domenico, for his guidance and invaluable support throughout this research journey. His advice and encouragement have been fundamental to the completion of this work.

On a personal level, I would like to express my deep gratitude to my parents, for their love, support, and patience. Your trust and encouragement have been my strength throughout this journey.

Finally, heartfelt thanks to my friends, for their friendship and for sharing with me both the most challenging and lightest moments of this experience.

To all those who have contributed, even indirectly, to my academic journey, my sincere recognition and appreciation.



”

*“Traveling — it leaves you speechless, then turns
you into a storyteller.”*

— **Ibn-Battuta**, *The Travels of Ibn Battutah*
(Maghrebi traveller, explorer and scholar)

ABSTRACT

This thesis embarks on a comprehensive exploration of the Travelling Salesman Problem (TSP), a cornerstone problem in the field of computational optimization, known for its straightforward premise yet complex resolution. At the core of this investigation lies a dual objective: to dissect the intricate nature of TSP through various algorithmic lenses and to introduce a variant of the Ant Colony Optimization (ACO) technique, termed the Red-Black Ant Colony System, aimed at enhancing solution efficiency and accuracy.

Keywords: Ant Colony System, Travelling Salesman, Metaheuristics, NP-hard problems, Approximation algorithms

SOMMARIO

Questa tesi intraprende un'esplorazione approfondita del Problema del Commesso Viaggiatore (TSP), un problema fondamentale nel campo dell'ottimizzazione computazionale, noto per la sua premessa semplice ma risoluzione complessa. Al centro di questa indagine c'è un duplice obiettivo: analizzare la natura intricata del TSP attraverso varie prospettive algoritmiche e introdurre una variante della tecnica di Ant Colony Optimization (ACO), denominata Red-Black Ant Colony System, mirata a migliorare l'efficienza e l'accuratezza della soluzione.

Parole Chiave: Ant Colony System, Travelling Salesman, Metaheuristics, NP-hard problems, Approximation algorithms

CONTENTS

List of Figures	xiii
List of Tables	xv
Glossary	xvii
Acronyms	xix
Symbols	xxi
1 Introduction	1
1.1 Basic Information	1
1.1.1 Definition and Importance of TSP	1
1.1.2 Historical Overview	1
1.2 The Problem	2
1.2.1 Mathematical Formulation	2
1.2.2 Integer Linear Programming Formulations	3
1.2.3 Variants and Applications	4
1.3 Thesis Objectives	5
1.3.1 Main Objectives	5
1.3.2 Theoretical Constraints	5
1.3.3 Practical Considerations	6
1.4 Thesis Structure	6
1.4.1 Chapter Overview	6
2 Theoretical Framework: Complexity and Efficiency	7
2.1 Computational Complexity	7
2.1.1 Time Complexity	7
2.1.2 Space Complexity	8
2.2 The Polynomial time (P) vs. Nondeterministic Polynomial time (NP) Problem	9
2.2.1 Definition and Relevance	9
2.2.2 Implications for the Travelling Salesman Problem (TSP)	10
2.3 Complexity of the TSP	10

2.3.1	Impact on Algorithm Design	10
3	Exact Algorithms for the TSP	13
3.1	Overview of Exact Algorithms	13
3.1.1	Brute-Force Method	13
3.1.2	Bellman-Held-Karp Algorithm	13
3.2	Analysis of Exact Algorithms	16
3.2.1	Computational Feasibility	16
3.2.2	Advantages and Disadvantages	16
3.2.3	Concorde	17
4	Heuristic and Metaheuristic Algorithms	19
4.1	Heuristic Approaches	19
4.1.1	Concept and Necessity	19
4.1.2	Greedy Algorithms	19
4.2	Local Search Algorithms	22
4.2.1	2-opt and 3-opt Techniques	22
4.2.2	Lin–Kernighan	23
4.3	Metaheuristic Algorithms	24
4.3.1	Simulated Annealing	25
4.3.2	Genetic Algorithms	26
4.3.3	Tabu Search	29
4.4	Ant Colony Optimization	31
4.4.1	Fundamental Principles of Ant Colony Optimization (ACO) . . .	32
4.4.2	Application to the TSP	32
4.4.3	Variants and Improvements	33
4.4.4	Advantages and Challenges	33
4.4.5	Conclusions	34
5	Ant Colony System by Dorigo et al	37
5.1	Fundamentals	37
5.1.1	Biological Inspiration	37
5.1.2	Algorithmic Principles	37
5.1.3	Application to TSP	38
5.2	Implementation Details	39
5.2.1	Pheromone Update Rules	39
5.2.2	Ant Movement Rules	39
5.2.3	Pseudocode	41
5.2.4	Complexity Analysis	41
5.2.5	Key Parameters and their Impact	41
5.3	Variants and Extensions	42
5.4	Applications beyond the TSP	42

6	Red-Black Ant Colony System	43
6.1	Introduction	43
6.2	The Red-Black Ant Colony System Algorithm (Red-Black Ant Colony System (RB-ACS))	43
6.2.1	Pheromone Initialization	43
6.2.2	Parallel Search with Red and Black Ant Groups	44
6.2.3	Differentiated Parameter Settings	44
6.2.4	Improved Global Pheromone Update	45
6.2.5	Pseudocode	45
6.3	Conclusion	45
7	Experimental Results	47
7.1	Experimental Methodology	47
7.1.1	Setup	47
7.1.2	Problem Instances	47
7.1.3	Implemented Algorithms	48
7.1.4	Evaluation Metrics	48
7.2	Results Analysis	48
7.2.1	General Performance	48
7.2.2	Detailed Analysis by Algorithm	48
7.2.3	Scalability Analysis	52
7.3	Discussion	53
8	Conclusions	55
8.1	Final Reflections	55
8.2	Concluding Remarks	56

LIST OF FIGURES

1.1	Two possible solutions for the (TSP) in a simplified instance of the United States, one shorter (in green) and one longer (in red).	2
2.1	Growth rate of different complexities	8
2.2	Classification of computational problems	9
3.1	Brute force TSP with four cities	14
3.2	Complexity of Exact Algorithms	16
3.3	Maximum problem size solvable in one hour (approximate)	17
4.2	2-opt swap	22
5.1	Ant behavior when facing an obstacle	38
6.1	An example of tours constructed by red and black ants in the RB-ACS algorithm.	44
7.1	Distribution of optimality gap per algorithm	49
7.2	Heatmap of optimality gap	49
7.3	Execution time by problem size	53
7.4	Gap by problem size	54
7.5	Comparison of top 3 algorithms: Gap	54

LIST OF TABLES

2.1	Number of possible permutations per number of cities	8
2.2	Comparison between different algorithms for TSP	11
4.1	Results of the Nearest Neighbor algorithm	22
4.2	Results of Nearest Neighbor algorithm with 2-opt optimization	23
4.3	Results of the Simulated Annealing algorithm	25
4.4	Results of the Genetic Algorithm	27
5.1	Results of the ACS algorithm	40
6.1	Results of the RB-ACS algorithm	46
7.1	Results NN vs NN+2Opt	50
7.2	Results SA vs SA2Opt	50
7.3	Results GA vs GA+2Opt	51
7.4	Results ACS vs ACS+2Opt	51
7.5	Results RB-ACS vs RB-ACS+2Opt	52

ACRONYMS

ACO	Ant Colony Optimization 31–34)
ACS	Ant Colony System 5, 6, 11, 33, 35, 37–45)
ASrank	Ant System with Rank-based Pheromone Update 42)
BnB	Branch and Bound 15, 16)
GA	Genetic Algorithm 11, 26)
LK	Lin-Kernighan 23, 24)
MMAS	Max-Min Ant System 33, 42)
MTZ	Miller-Tucker-Zemlin 3)
NNS	Nearest Neighbor Search 20, 21)
NP	Nondeterministic Polynomial time 7, 9–11)
OX	Order Crossover 29)
P	Polynomial time 9, 10)
PMX	Partially Mapped Crossover 29)
RB-ACS	Red-Black Ant Colony System 5, 6, 43–45)
SA	Simulated Annealing 25)
TS	Tabu Search 29–31)
TSP	Travelling Salesman Problem 1, 2, 4–11, 13–34, 38, 41–43, 45)

INTRODUCTION

1.1 Basic Information

1.1.1 Definition and Importance of TSP

The importance of the TSP goes beyond its seemingly simple formulation, branching into various fields and applications. In logistics and transportation, solutions to the TSP enable optimal routing capable of leading to significant cost savings and efficiency improvements. In manufacturing, solution algorithms for TSP are used to optimize drill movements in printed circuit board production or robotic arms in assembly lines to reduce production times. Beyond these, algorithms for TSP are employed in DNA sequencing, astronomy, and even in the creation of works of art, showing its versatility and importance in solving real-world problems.

1.1.2 Historical Overview

The Travelling Salesman Problem (TSP) represents one of the most studied challenges in computational mathematics and optimization, with a history that intertwines with the development of mathematical and operational research. Its evolution from early conceptual explorations to a crucial problem in algorithm design is a testament to its complexity and wide applicability. The study of the TSP can be traced back to the works of Sir William Rowan Hamilton and Thomas Penyngton Kirkman in the 19th century, focused on Hamiltonian cycles and paths. Karl Menger, in the 1920s, introduced the "Messenger Problem", laying the foundation for the modern formulation of the TSP and emphasizing the search for efficient routing.

The advent of computers and operational research techniques marked a significant phase in TSP research. Early pioneers such as George Dantzig and Delbert Ray Fulkerson

applied linear programming and cutting plane methods, illustrating the computational complexity of the problem and its potential for practical applications.

In the 1940s, the TSP began to find relevance in agriculture and statistics, with researchers using the problem's solution algorithms to optimize data collection and survey processes, showing the versatility of the TSP beyond theoretical mathematics.

In recent decades, the exploration of heuristic and bio-inspired algorithms, particularly the work of Marco Dorigo on combinatorial optimization through ant colonies, has opened new avenues for solving the TSP, highlighting continuous innovation in addressing its NP-hardness.

1.2 The Problem

The Travelling Salesman Problem: Given a set of cities and the distances between each pair of cities, find the shortest possible route that visits each city exactly once and returns to the original city.

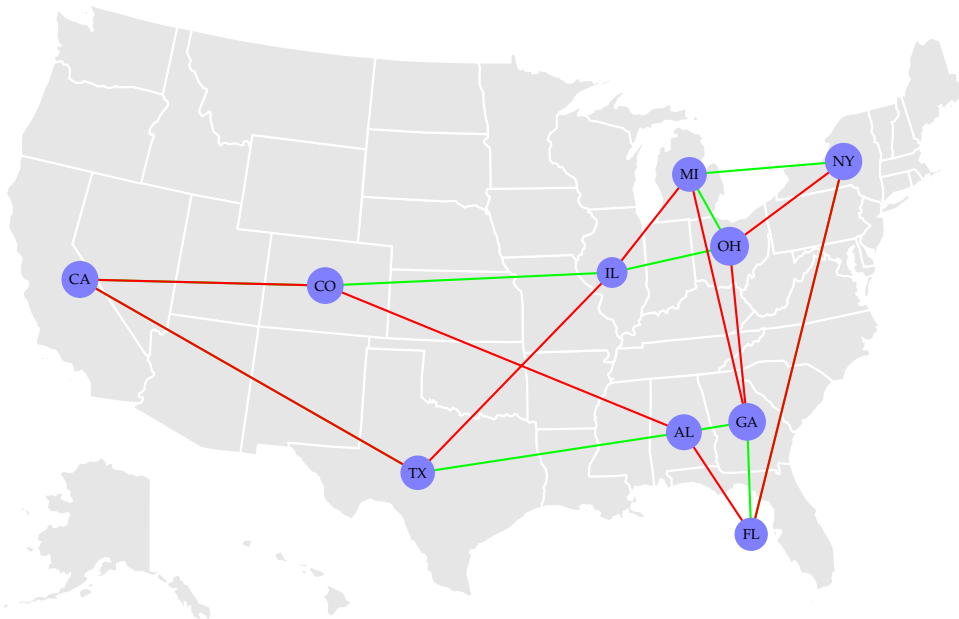


Figure 1.1: Two possible solutions for the (TSP) in a simplified instance of the United States, one shorter (in green) and one longer (in red).

1.2.1 Mathematical Formulation

Given a graph $G = (V, E)$, where V is the set of vertices (cities) and E is the set of edges (paths between cities), with each edge $(i, j) \in E$ assigned a weight w_{ij} representing the cost of traveling from city i to city j , the TSP seeks a Hamiltonian cycle of minimum weight.

The objective function to minimize is:

$$\min \sum_{i=1}^n \sum_{j \neq i, j=1}^n w_{ij} x_{ij},$$

subject to constraints:

$$\sum_{i=1, i \neq j}^n x_{ij} = 1, \quad \forall j \in V,$$

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \quad \forall i \in V,$$

together with subtour elimination constraints, ensuring that each city is visited exactly once and the tour is cyclic.

1.2.2 Integer Linear Programming Formulations

1.2.2.1 Miller–Tucker–Zemlin Formulation

The Miller-Tucker-Zemlin (MTZ) formulation introduces auxiliary variables u_i for ordering, along with binary variables x_{ij} :

Objective:

$$\min \sum_{i=1}^n \sum_{j \neq i, j=1}^n c_{ij} x_{ij},$$

Subject to:

$$\sum_{i=1, i \neq j}^n x_{ij} = 1, \quad \forall j,$$

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \quad \forall i,$$

$$u_i - u_j + 1 \leq (n-1)(1 - x_{ij}), \quad \forall 2 \leq i \neq j \leq n,$$

$$2 \leq u_i \leq n, \quad \forall 2 \leq i \leq n.$$

This formulation enforces a single tour through auxiliary ordering variables, effectively preventing subtours.

1.2.2.2 Dantzig–Fulkerson–Johnson Formulation

The DFJ formulation, known for its efficiency, directly introduces subtour elimination constraints:

Objective:

$$\min \sum_{i=1}^n \sum_{j \neq i, j=1}^n c_{ij} x_{ij},$$

Subject to:

$$\begin{aligned} \sum_{i=1, i \neq j}^n x_{ij} &= 1, \quad \forall j, \\ \sum_{j=1, j \neq i}^n x_{ij} &= 1, \quad \forall i, \\ \sum_{i \in S, j \notin S} x_{ij} &\geq 1, \quad \forall S \subset V, S \neq \emptyset, S \neq V. \end{aligned}$$

This formulation combats subtours by ensuring that at least one edge exits from every subset of vertices, guaranteeing a single connected tour.

1.2.3 Variants and Applications

The TSP adapts to a wide range of practical challenges, demonstrating its versatility. Key variants include:

- **Symmetric vs. Asymmetric TSP (sTSP vs. aTSP):** sTSP implies that the weight of an edge between two nodes is equal regardless of the direction of travel, while aTSP allows distinct weights, reflecting the complexity of real-world routes.
- **Euclidean TSP:** This variant places cities in a Euclidean plane, emphasizing direct and straight-line distances, fundamental for drone navigation and infrastructure planning.
- **TSP with Time Windows:** Adds specific time intervals within which each city must be visited, crucial for planning in delivery services and maintenance operations.
- **Vehicle Routing Problem (VRP):** An extension of the TSP that involves multiple vehicles, central to optimizing fleet logistics and distribution networks.

The application of the TSP and its variants extends to various fields, showing its utility in addressing complex optimization problems:

- **Logistics Optimization:** The Euclidean TSP and VRP are essential for reducing the cost of delivery routes and improving logistics efficiency.
- **Manufacturing:** The TSP helps optimize production processes, reducing downtime and improving overall workflows.
- **Planning and Scheduling:** The TSP with Time Windows ensures timely operations, optimizing resource allocation for services with critical timing requirements.
- **Network Design:** TSP algorithms facilitate the development of cost-effective and efficient telecommunication and transportation networks.
- **Genomics:** In genetics, the TSP supports advances in genome sequencing, offering insights into complex biological structures and processes.

1.3 Thesis Objectives

This thesis aims to explore the Travelling Salesman Problem (TSP) from both a theoretical and practical perspective. The focus is on examining the Red-Black Ant Colony System RB-ACS, a variant of the Ant Colony System algorithm Ant Colony System (ACS), with some modifications, to understand its effectiveness in addressing the challenges of the TSP. The study has two objectives: improving the understanding of the TSP in computational optimization and evaluating the performance of the modified RB-ACS compared to conventional solutions.

1.3.1 Main Objectives

The thesis is guided by the following objectives:

- Examine the existing body of work on the TSP, gathering its mathematical foundations, evolution, and the set of strategies previously devised for its resolution.
- Detail the operation of the RB-ACS, emphasizing its theoretical foundations, operational mechanics, and its distinctions from more traditional approaches.
- Conduct a systematic experimental comparison of the RB-ACS against standard TSP solution algorithms across a range of problem sets, aiming to discern its operational effectiveness and scalability.
- Identify practical scenarios where the modified algorithm could be applied, showing its relevance and potential benefits in tangible contexts.

1.3.2 Theoretical Constraints

The exploration within this thesis is tailored to fit the available computational resources, focusing on TSP instances that balance presenting a notable challenge and the feasibility of analysis within the constraints of our computational setup. A notable aspect of this research concerns the reimplementation in Rust of all algorithms discussed, due to the unavailability of the original source code for some of these algorithms. This necessity highlights a possible variation in performance metrics, which might not fully align with those derived from the authors' original implementations. Such differences could stem from various factors, including the intrinsic characteristics of Rust as a programming language and the complexities involved in translating complex algorithmic logic into code. Consequently, while aiming to provide a comparative analysis of algorithmic performance, it is imperative to consider these reimplementation factors as potential variables influencing the results.

Furthermore, while acknowledging the potential for real-world applications of the algorithms under study, in-depth case studies that go beyond theoretical exploration and computational experimentation fall outside the scope of this bachelor's thesis. The

main focus remains on the theoretical foundations and computational evaluation of TSP solutions.

1.3.3 Practical Considerations

The practicality of this thesis is influenced by several limitations that guide its experimental approach and evaluation methodology:

- The performance evaluation of the RB-ACS is conducted through a carefully chosen subset of TSP instances. These instances aim to provide a diverse but not exhaustive representation of potential problem scenarios, highlighting the intrinsic limitation in capturing the entire spectrum of TSP configurations.
- Due to computational resource constraints, the scalability of the proposed algorithm is tested in a constrained environment. This limitation may hinder the extrapolation of performance results to significantly larger or more complex TSP instances.
- The thesis considers the practical implications of its findings in a theoretical and computational context, without delving into field testing or specific case studies for applications.

1.4 Thesis Structure

1.4.1 Chapter Overview

Chapter 1 introduces the Travelling Salesman Problem (TSP), covering what it is, why it is important, and a bit of its history. It outlines the main questions this thesis aims to answer and briefly outlines the methods used. Chapter 2 analyzes the overall framework of computational complexity, focusing on the famous P vs. NP question, to help understand why solving the TSP can be so challenging. The discussion continues in Chapters 3 and 4 to explore different ways of approaching the TSP. Chapter 3 examines exact solutions while Chapter 4 discusses heuristic and metaheuristic methods, including well-known strategies such as genetic algorithms and simulated annealing. Chapter 5 focuses on the ACS, explaining how it works, why it is nature-inspired, and how it applies to solving the TSP. Then, Chapter 6 introduces a variant of the ACS, the RB-ACS. This chapter explains how it was developed, the new ideas it brings, and how tests demonstrate that it could offer an advantage in solving TSP instances. Chapter 7 presents the experimental results obtained. Finally, Chapter 8 concludes the thesis by summarizing the highlights of this thesis. This thesis aims to contribute to the conversation on solving the TSP, seeking to bridge the gap between theoretical challenges and real-world solutions, and opening the door to further investigations into optimization problems.

THEORETICAL FRAMEWORK: COMPLEXITY AND EFFICIENCY

2.1 Computational Complexity

Computational complexity refers to the study of resource requirements of algorithms, focusing primarily on time and space resources as a function of input size. This field categorizes problems based on the minimum resources needed to solve them, providing a theoretical framework for understanding the intrinsic difficulty of computational problems and the efficiency of algorithms designed to solve them. [0]

2.1.1 Time Complexity

Time complexity is a measure of the amount of computational time an algorithm takes to complete as a function of the input length. It is often expressed using Big O notation, which describes the upper bound of the growth rate of execution time [0]. Understanding the time complexity of an algorithm is crucial for predicting its scalability and feasibility in practical applications, especially for problems known to be computationally intensive such as the TSP.

The TSP, being NP-hard, does not have a known polynomial-time solution that can efficiently solve all instances of the problem. The brute-force approach for the TSP, for example, has a factorial time complexity ($O(n!)$), making it computationally infeasible even for moderately large problem instances. This has led to the exploration of various heuristic and metaheuristic algorithms that aim for acceptable solutions in polynomial time ($O(n^k)$), where k is a constant, trading accuracy for efficiency and practical applicability. [0]

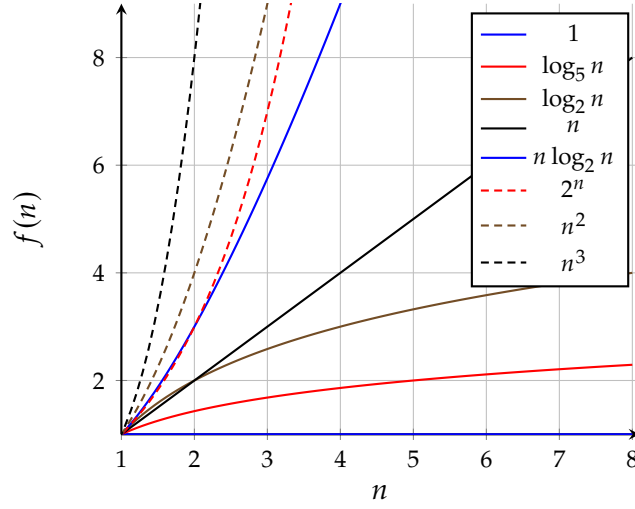


Figure 2.1: Growth rate of different complexities

Table 2.1: Number of possible permutations per number of cities

Number of cities	Number of Permutations
4	24
5	120
6	720
7	5,040
8	40,320
9	362,880
10	3,628,800
11	39,916,800
12	479,001,600
13	6,227,020,800
14	87,178,291,200
15	1,307,674,368,000
16	20,922,789,888,000
17	355,687,428,096,000
18	6,402,373,705,728,000
19	121,645,100,408,832,000
20	2,432,902,008,176,640,000
25	15,511,210,043,330,985,984,000,000

2.1.2 Space Complexity

Space complexity measures the total amount of memory an algorithm requires as a function of input size. Like time complexity, space complexity is crucial for evaluating the efficiency of an algorithm, particularly in scenarios where memory resources are limited. In the context of the TSP and similar optimization problems, the space complexity of an algorithm can significantly impact its usability in real-world applications, where solutions often need to be computed in real-time or on hardware with limited memory capacity. [0].

For example, dynamic programming approaches for the TSP, such as the Held-Karp

algorithm, offer a more favorable time complexity compared to brute force ($O(n^2 2^n)$) but at the cost of exponential space complexity ($O(n^2)$). Such trade-offs between time and space complexity are central considerations in algorithm design, especially when developing new algorithms intended for large-scale problem instances, such as those encountered in logistics and routing applications.

2.2 The P vs. NP Problem

The P vs NP problem represents one of the most significant open questions in the field of theoretical computer science. It concerns establishing whether every problem whose solution can be efficiently verified (in polynomial time) by a computer can also be solved just as quickly (also in polynomial time). The P class includes problems that can be solved quickly, while the NP class comprises problems for which a given solution can be verified quickly. [0].

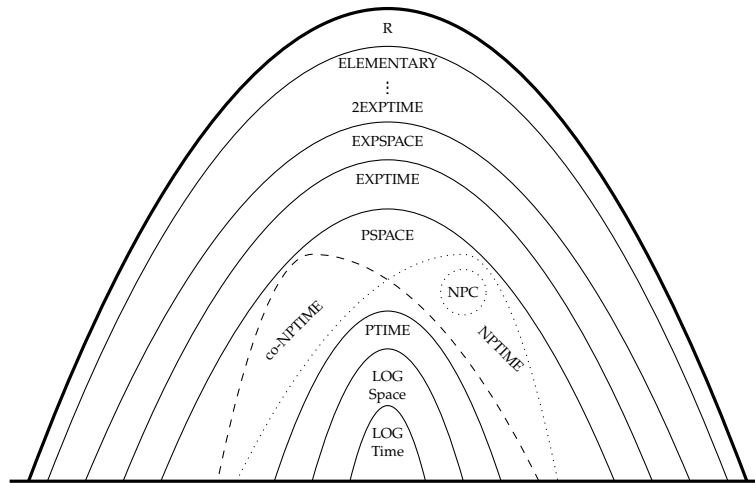


Figure 2.2: Classification of computational problems

2.2.1 Definition and Relevance

Formally, a problem belongs to the P class if it can be solved in polynomial time, which means that the amount of time needed to solve the problem grows polynomially with the size of the input. The NP class, on the other hand, comprises problems for which a given solution can be verified in polynomial time. The P vs. NP problem essentially asks whether these two classes are equal; that is, whether every problem that can be verified quickly can also be solved quickly.

The relevance of the P vs. NP problem lies in its implications for a vast field of disciplines, from cryptography and algorithm design to decision-making processes and beyond. A proof that P equals NP could potentially revolutionize the fields of mathematics and computer science, enabling efficient solutions to a multitude of complex problems

currently considered intractable. Conversely, proving that P does not equal NP would formalize the intrinsic computational difficulty of these problems.

2.2.2 Implications for the TSP

The classification of the TSP as NP-hard underscores the complexity and computational challenges associated with finding an exact solution for large problem instances. This classification has significant implications for both theoretical research and practical applications in the field. It implies that unless P equals NP , one should not expect to discover an algorithm capable of solving quickly (in polynomial time) and accurately every instance of the TSP. This awareness has shifted the focus of much of the research on the TSP toward the development of heuristic and metaheuristic algorithms, which aim to find sufficiently good solutions in a reasonable time frame, rather than pursuing the elusive goal of an exact solution for all possible instances. [0].

Furthermore, the implications of the TSP being NP-hard extend beyond algorithm development. It influences how problems are modeled in practical scenarios where TSP-like problems arise, such as logistics and routing, network design, and scheduling. In these applications, the emphasis is often on finding solutions that approximate the optimal but can be obtained much more quickly than an exact algorithm would allow. This approach enables industry to achieve efficiency and cost savings even when facing complex routing and scheduling problems.

Therefore, the implications of the TSP classification resonate in both the academic study of algorithm theory and in practical considerations on applying these theories in real-world situations.

2.3 Complexity of the TSP

The proof that the TSP is NP-hard was first presented by Richard M. Karp in 1972. Karp's work, part of a paper that identified 21 NP-complete problems, laid the foundation for understanding the computational limits of algorithmic solutions for the TSP and similar problems. By demonstrating that the decision version of the TSP (determining whether a tour of a given length or less exists) is NP-complete, Karp showed that the optimization version of the TSP, where the goal is to find the shortest possible tour, is NP-hard.

Karp's proof employs a technique known as reduction, in which a known NP-complete problem is transformed into an instance of the TSP in polynomial time. This method illustrates that if a polynomial-time algorithm could solve the TSP, it could, by extension, solve all problems in NP, a proposition that remains unverified.

2.3.1 Impact on Algorithm Design

The NP-hardness of the TSP has a profound impact on the design of algorithms to solve it, fostering the development of various strategies aimed at overcoming computational

Algorithm	Time - Space Complexity	Description	Advantages	Disadvantages
Held-Karp	$O(n^2 2^n) - O(n^2)$	Exact algorithm using dynamic programming	Provides optimal solution	Infeasible for large n
Nearest Neighbor	$O(n^2) - O(n)$	Greedy heuristic algorithm	Simple and fast	Often far from optimal
Christofides	$O(n^3) - O(n^2)$	Approximation algorithm with 3/2 ratio	Best approximation for metric TSP	Still not exact
Genetic algorithms	$O(n^2) - O(n)$	Metaheuristic algorithm inspired by natural selection	Flexible and adaptable	No guarantee of optimal solution

Table 2.2: Comparison between different algorithms for TSP

obstacles:

- **Exact algorithms:** Despite the NP-hardness of the TSP, exact algorithms have been devised, such as the Held-Karp algorithm. These algorithms guarantee an optimal solution, but do so at a computational cost that increases exponentially with problem size, making them impractical for large instances.
- **Heuristic approaches:** To tackle larger TSP instances, heuristic algorithms are employed. These algorithms do not guarantee an optimal solution, but can often find good solutions in a fraction of the time required by exact methods. Examples include the Nearest Neighbor algorithm and Christofides' algorithm.
- **Metaheuristic algorithms:** For even more complex instances, metaheuristic approaches, such as the Genetic Algorithm (GA) and the Ant Colony System (ACS), offer flexible strategies that explore the solution space more broadly, aiming to escape local optima and approach global optima more effectively. These techniques draw inspiration from natural processes and have the advantage of adaptability to different problem instances.

EXACT ALGORITHMS FOR THE TSP

This chapter provides an in-depth examination of exact algorithms for solving the TSP, incorporating detailed pseudocode to clarify the operational principles of each method. Through these algorithms, we explore the computational landscape of finding optimal solutions to the TSP.

3.1 Overview of Exact Algorithms

Exact algorithms for the TSP are characterized by their ability to invariably find the optimal solution, albeit with computational costs that can become prohibitive as the number of cities increases. These algorithms serve as a theoretical and practical foundation for understanding the limits of computational resolution of the TSP.

3.1.1 Brute-Force Method

The brute-force method systematically examines every possible tour to identify the one with the shortest total distance. Despite its simplicity, the exponential growth of calculations makes it impractical for non-trivial sizes.

The computational complexity of this method is $O(n!)$, which reflects the factorial number of tours that must be evaluated.

3.1.2 Bellman-Held-Karp Algorithm

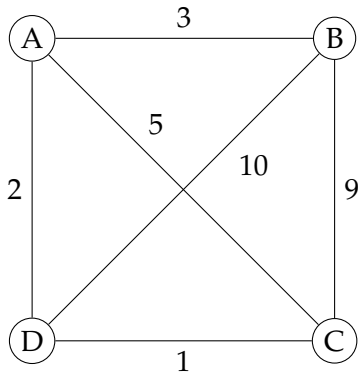
The Bellman-Held-Karp algorithm represents a significant advancement in exact solutions for the TSP by exploiting dynamic programming to reduce the computational burden. This approach computes the shortest path by dividing the problem into smaller subproblems and storing the results of these subproblems to avoid redundant calculations exploiting dynamic programming.

Algorithm 1 TSP Brute Force

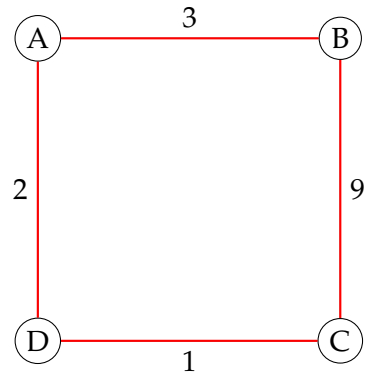
```

1: procedure BRUTEFORCETSP(cities)
2:    $min\_distance \leftarrow \infty$ 
3:    $min\_tour \leftarrow \emptyset$ 
4:   for all tours  $t$  of cities do
5:      $distance \leftarrow \text{TOURDISTANCE}(t)$ 
6:     if  $distance < min\_distance$  then
7:        $min\_distance \leftarrow distance$ 
8:        $min\_tour \leftarrow t$ 
9:     end if
10:  end for
11:  return  $min\_tour$ 
12: end procedure

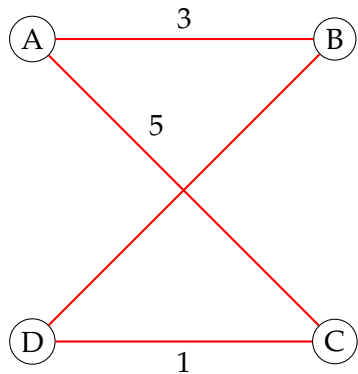
```



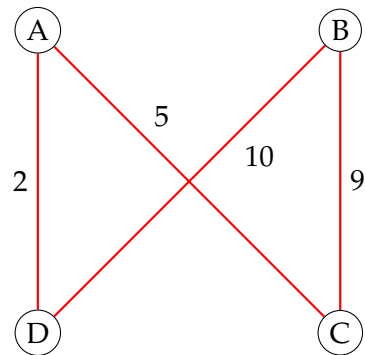
(a) Starting graph with four cities



(b) Optimal path with total distance 15



(c) Alternative path with total distance 19



(d) Alternative path with total distance 26

Figure 3.1: Brute force TSP with four cities

Algorithm 2 Bellman-Held-Karp Algorithm

```

1: procedure BELLMANHELDKARP(cities)
2:   Create a table distances to store the shortest paths
3:   Initialize all entries in distances to  $\infty$ 
4:   distances[1][{1}]  $\leftarrow$  0 ▷ Starting city
5:   for  $m = 2$  to  $|cities|$  do
6:     for all subsets  $S$  of cities of size  $m$  containing 1 do
7:       for all  $j \in S, j \neq 1$  do
8:         distances[ $j$ ][ $S$ ]  $\leftarrow \min_{k \neq j, k \in S} (\text{distances}[k][S \setminus \{j\}] + \text{distance}(k, j))$ 
9:       end for
10:    end for
11:  end for
12:  return  $\min_j (\text{distances}[j][cities] + \text{distance}(j, 1))$ 
13: end procedure

```

The Bellman-Held-Karp algorithm operates with a time complexity of $O(n^2 \cdot 2^n)$ 3.2, making it significantly more efficient than the brute-force method for moderate-sized problem instances. However, the exponential component of its complexity still limits its practical applicability to relatively small instances of the TSP.

3.1.2.1 Branch and Bound

Branch and Bound (Branch and Bound (BnB)) is a general algorithmic technique that can be applied to various combinatorial optimization problems, including the TSP. It systematically explores the solution space by dividing it into smaller subsets (branching) and using bounds to eliminate subsets that do not contain the optimal solution, thus reducing the search space.

Algorithm 3 BnB for the TSP

```

1: procedure BRANCHANDBOUNDTSP(cities)
2:   Initialize a priority queue  $Q$  with a partial solution starting from the first city
3:   min_distance  $\leftarrow \infty$ 
4:   while  $Q$  is not empty do
5:     Take the partial solution with the lowest lower bound from  $Q$ 
6:     if it represents a complete tour then
7:       if its cost is less than min_distance then
8:         Update min_distance and record the tour
9:       end if
10:    else
11:      Branch by adding a city to the partial solution
12:      Calculate bounds for the new partial solutions
13:      Add the new partial solutions to  $Q$ 
14:    end if
15:  end while
16:  return min_distance
17: end procedure

```

The efficiency of BnB for the TSP depends significantly on the bounding function used to estimate the lower bound of partial solutions. An effective bound can lead to substantial reductions in the search space, allowing the algorithm to find the optimal solution more quickly than exhaustive search methods. However, its worst-case time complexity remains $O(n!)$ 3.2, which means that for very large problem instances, even this sophisticated approach may not be practical.

3.2 Analysis of Exact Algorithms

3.2.1 Computational Feasibility

Exact algorithms for the TSP, such as the brute-force method, dynamic programming, and the Bellman-Held-Karp algorithm, offer theoretical guarantees for finding the optimal tour. However, their computational feasibility is significantly compromised as the number of cities increases. The factorial growth rate of the brute-force method's complexity and the exponential growth rate of the Bellman-Held-Karp complexity limit their practical application to relatively small instances of the TSP.

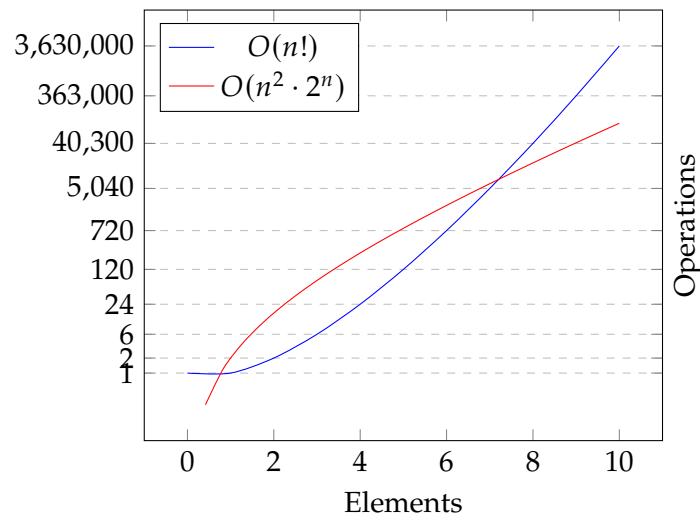


Figure 3.2: Complexity of Exact Algorithms

3.2.2 Advantages and Disadvantages

In summary:

Advantages:

- Guarantee of finding an optimal solution.
- Provide a benchmark for evaluating the performance of heuristic and metaheuristic algorithms.

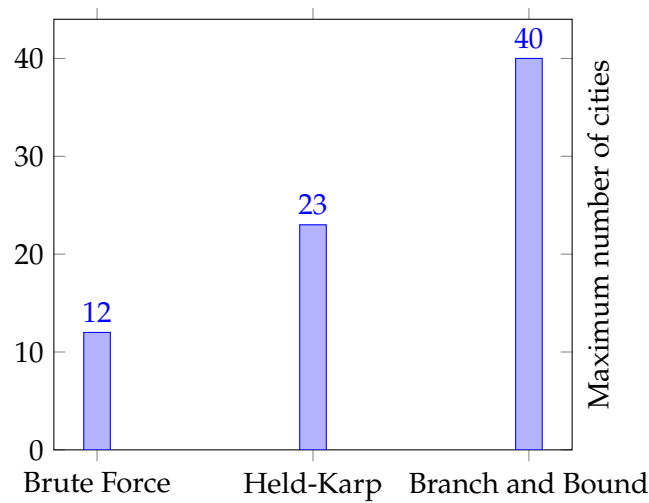


Figure 3.3: Maximum problem size solvable in one hour (approximate)

Disadvantages:

- Exponential time complexity that makes them impractical for large instances.
- Significant computational resources required even for moderate-sized problems.
- Dynamic programming and Held-Karp, while more efficient than the brute-force method, still face limitations due to their space complexity and the need for extensive computation.

3.2.3 Concorde

Concorde is considered one of the most efficient exact solvers for solving the TSP. Developed by a group of researchers led by Applegate et al., this solver exploits a combination of advanced methods such as branch-and-bound, cutting-plane, and linear and combinatorial programming techniques [0]. Concorde is capable of solving TSP instances with thousands of cities in reasonable times, making it a reference tool for experimental comparisons and large-scale practical applications.

Concorde's efficiency is due to its ability to drastically reduce the search space through branch-and-bound, enhanced with the use of cutting planes to refine the lower bounds of solutions. In addition, Concorde implements effective heuristics to identify good initial solutions, which help limit the exploration of suboptimal solutions. Concorde has also been used to solve benchmark problems in the TSP literature, such as the TSPLIB problem instances [0], demonstrating its effectiveness for real-world problem sizes [0].

Advantages:

- Ability to solve large TSP instances.
- Combined use of advanced optimization techniques.

- Efficient and well-documented implementations.

Disadvantages:

- Requires high computational resources for extremely large instances.
- The complexity of the implemented techniques can make it difficult to adapt to problems slightly different from the TSP.

The exploration of exact algorithms lays the foundation for understanding the intrinsic computational complexities of the TSP. Although their practical application is limited by their computational demands, they remain crucial for theoretical analysis and for setting the stage for the alternative solution techniques discussed in subsequent chapters.

HEURISTIC AND METAHEURISTIC ALGORITHMS

4.1 Heuristic Approaches

Heuristic approaches to solving problems such as the TSP are strategies designed to find sufficiently good solutions within a reasonable time, especially when an exact solution is not feasible due to the complexity [0] of the problem. These methods are essential in cases where the computational cost of finding the optimal solution is prohibitive, as is common with NP-hard problems such as the TSP [0].

4.1.1 Concept and Necessity

The concept of heuristic approaches is based on the idea of making educated guesses or following intuitive algorithms that aim to find solutions that are close to the best possible, without necessarily guaranteeing optimality [0]. Heuristics are characterized by their empirical strategies that make them significantly faster than exact methods in practice, albeit at the cost of potentially missing the optimal solution.

The necessity of heuristics stems from the computational intractability of many real-world problems [0]. Heuristics offer a pragmatic alternative by providing solutions that, while not guaranteed to be optimal, are sufficiently accurate for practical purposes and can be obtained much more quickly [0].

4.1.2 Greedy Algorithms

Greedy algorithms constitute a significant class of heuristic approaches characterized by their strategy of making the locally optimal choice at each stage with the hope of finding a global optimum [0]. In the context of the TSP and similar optimization problems, greedy algorithms simplify decision-making processes by breaking down a problem into

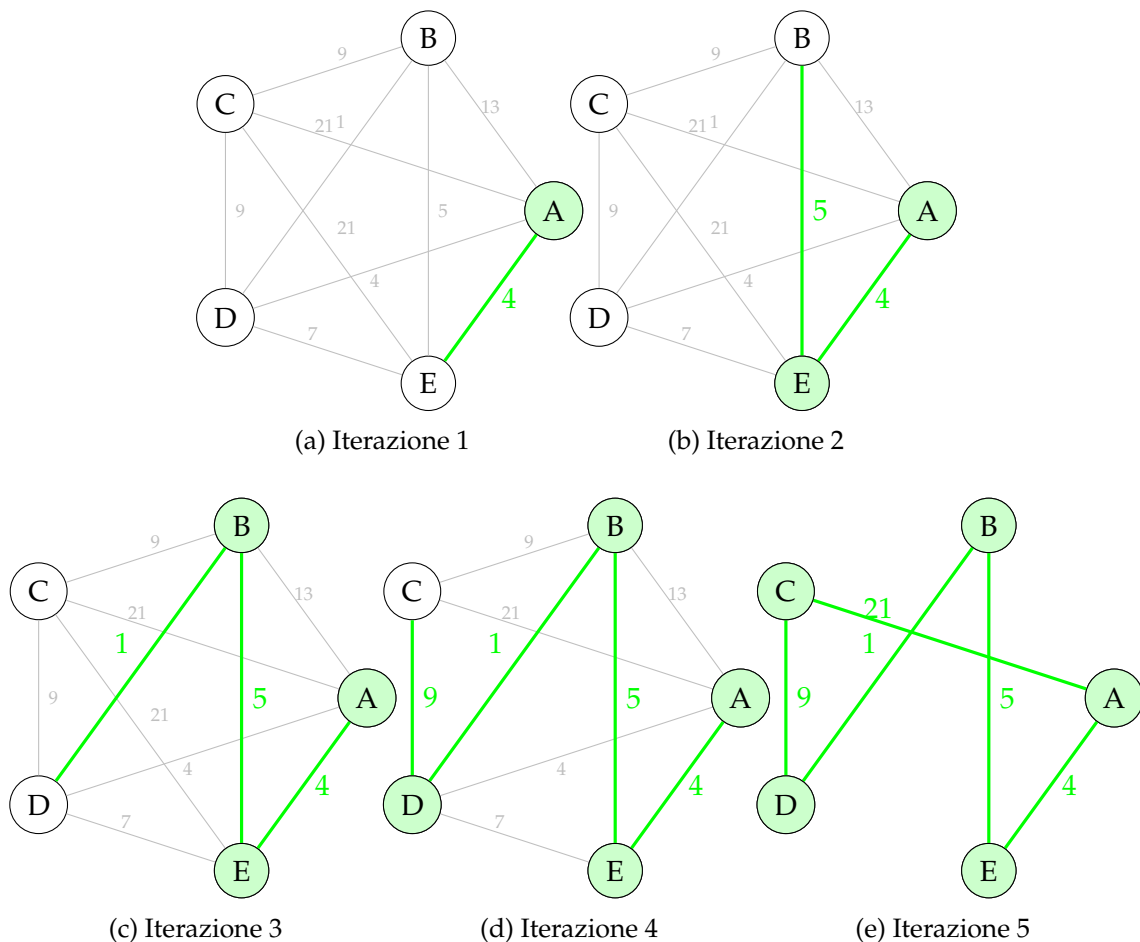
a series of steps and choosing the best available option at each step without considering the broader consequences of these choices[0].

4.1.2.1 Definition and Principle

A greedy algorithm constructs a solution piece by piece, always choosing the next piece that offers the most immediate benefit[0]. This approach is simple and straightforward, making it an attractive option for many problems where a quick and easy solution is more valuable than an optimal one. The effectiveness of greedy algorithms varies widely from problem to problem; in some cases, they achieve an optimal solution, while in others, they may only achieve a suboptimal solution[0].

4.1.2.2 Application to the TSP: Nearest Neighbor Search

In the TSP, a classic application of the greedy algorithm is the Nearest Neighbor Search (NNS), where the algorithm constructs a tour starting from an arbitrary city and, at each step, extends the tour by moving to the nearest unvisited city until all cities are visited[0].



Algorithm 4 TSP NNS

```

1: procedure NEARESTNEIGHBORSEARCH(cities, distances)
2:   start  $\leftarrow$  select an arbitrary city from cities
3:   current  $\leftarrow$  start
4:   tour  $\leftarrow$  list containing start
5:   unvisited  $\leftarrow$  cities  $\setminus$  {start}
6:   while unvisited  $\neq \emptyset$  do
7:     next  $\leftarrow$  city in unvisited with minimum distance from current
8:     add next to tour
9:     remove next from unvisited
10:    current  $\leftarrow$  next
11:  end while
12:  add start to tour to close the cycle
13:  return tour
14: end procedure

```

Pseudocode This greedy strategy ensures that at each step, the tour grows by adding the nearest possible destination, optimizing for the immediate next step without considering the overall tour length. While this method does not guarantee finding the shortest possible tour, it significantly reduces computation time compared to exhaustive search methods.

4.1.2.3 Advantages and Limitations

Advantages:

- *Simplicity*: Greedy algorithms are simple to implement and understand.
- *Efficiency*: They often provide solutions quickly, making them suitable for problems where speed is crucial.

Limitations:

- *Optimality*: Greedy choices do not always lead to the optimal solution, especially for complex problems like the TSP.
- *Short-sightedness*: By focusing on local optimality, they might overlook better solutions that require initially non-optimal choices.

Greedy algorithms, with their inherent simplicity and efficiency, play a crucial role in the heuristic resolution of problems, especially in cases where obtaining an exact solution is computationally impractical. Their application to the TSP highlights the balance between computational efficiency and solution quality, a theme that resonates through the study of heuristic and metaheuristic algorithms.

Table 4.1: Results of the Nearest Neighbor algorithm

Instance	Time (ms)	Tour Length	Optimal Length	Gap
berlin52	14	8980.92	7542.00	19.08
eil51	15	513.61	426.00	20.57
eil76	17	711.99	538.00	32.34
lin105	43	20362.76	14379.00	41.61
pr124	136	69299.43	59030.00	17.40
d198	183	18620.07	15780.00	18.00
lin318	466	54033.58	42029.00	28.56
u574	1295	46881.87	36905.00	27.03
fl1577	9547	27940.91	22249.00	25.58
rl5915	236473	707498.63	565530.00	25.10

4.2 Local Search Algorithms

Local search algorithms represent a class of heuristic methods designed to explore the solution space of an optimization problem by iteratively moving from one solution to a neighboring solution in the search space. These algorithms are particularly effective for problems such as the TSP. Local search provides a mechanism to improve an initial solution through small localized modifications [0, 0].

4.2.1 2-opt and 3-opt Techniques

The 2-opt and 3-opt techniques are specific local search strategies used to refine TSP solutions. They work by iteratively removing two or three edges from the tour and reconnecting the segments in a different order, aiming to reduce the total tour length with each operation.

4.2.1.1 2-opt Technique

The 2-opt algorithm systematically checks each pair of edges in the tour and determines whether swapping them would lead to a shorter path. This process is repeated until no further improvements can be made.

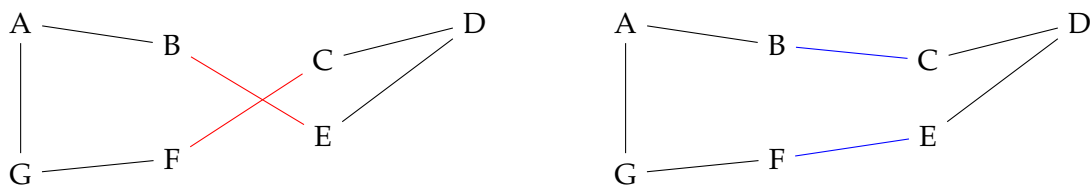


Figure 4.2: 2-opt swap

Algorithm 5 2-opt Technique for TSP

```

1: procedure TwoOpt(tour, distances)
2:   improved  $\leftarrow$  true
3:   while improved do
4:     improved  $\leftarrow$  false
5:     for  $i \leftarrow 1$  to  $\text{length}(\text{tour}) - 1$  do
6:       for  $j \leftarrow i + 1$  to  $\text{length}(\text{tour})$  do
7:         if TwoOptSwap(tour,  $i, j$ ) < TourDistance(tour) then
8:           tour  $\leftarrow$  TwoOptSwap(tour,  $i, j$ )
9:           improved  $\leftarrow$  true
10:        end if
11:      end for
12:    end for
13:  end while
14:  return tour
15: end procedure

```

Table 4.2: Results of Nearest Neighbor algorithm with 2-opt optimization

Instance	Time (ms)	Tour Length	Optimal Length	Gap
berlin52	124	8060.65	7542.00	6.88
eil51	284	440.90	426.00	3.50
eil76	313	599.05	538.00	11.35
lin105	2959	16199.70	14379.00	12.66
pr124	3268	62757.01	59030.00	6.31
d198	4443	16165.31	15780.00	2.44
lin318	9562	46408.41	42029.00	10.42
u574	26923	39896.00	36905.00	8.10
fl1577	200151	24214.30	22249.00	8.83
rl5915	6327036	620822.08	565530.00	9.78

4.2.1.2 3-opt Technique

The 3-opt technique extends the idea of 2-opt by considering three edges for reconnection. This allows a wider range of rearrangements at each step, potentially leading to more significant improvements at the cost of greater computational complexity.

Pseudocode for the 3-opt Technique is conceptually similar to 2-opt but involves more conditions for swaps, reflecting the greater complexity of considering three edges at a time.

4.2.2 Lin-Kernighan

The Lin-Kernighan algorithm (Lin-Kernighan (LK)) is one of the most advanced and powerful local search techniques for solving the TSP. Proposed by Shen Lin and Brian W. Kernighan in 1973, this algorithm extends the ideas of the k -opt technique, making

it more flexible and adaptable. The algorithm is not limited to a fixed size k of swaps, but dynamically decides the number of edges to remove and reconnect, making it very effective in improving initial solutions.[0]

Algorithm 6 Lin-Kernighan Algorithm for the TSP

```

1: procedure LINKERNIGHAN( $T$ )                                ▶  $T$  is the initial tour
2:    $T' \leftarrow T$                                            ▶ Initialize  $T'$  with  $T$ 
3:   repeat
4:     for each city  $x_1$  in  $T'$  do
5:        $k \leftarrow 2$ 
6:       repeat
7:         Select a set of  $k$  edges  $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$  from  $T'$ 
8:         Generate a new tour  $T''$  by removing edges and reconnecting according
           to the  $k$ -opt move
9:         if  $T''$  is shorter than  $T'$  then
10:            $T' \leftarrow T''$ 
11:           break                                             ▶ Exit inner loop if there is improvement
12:         end if
13:          $k \leftarrow k + 1$ 
14:       until No further improvement
15:     end for
16:   until All cities have been tried as starting cities
17:   return  $T'$ 
18: end procedure

```

4.2.2.1 Description

The LK algorithm starts with an initial solution and attempts to iteratively improve this solution through a series of generalized k -opt moves. Each k -opt move seeks to reduce the total tour length by exchanging k edges of the current tour with new edges, thus improving the overall path.

The LK algorithm has been extensively studied and improved over the years. For a detailed analysis of the algorithm and its extensions, the following references can be consulted [0, 0, 0].

Local search algorithms, including 2-opt and 3-opt techniques, offer powerful methods for improving TSP solutions. By iteratively refining an initial solution, they balance exploration of the solution space and exploitation of known good configurations, providing effective means to approach optimal solutions for complex optimization problems.

4.3 Metaheuristic Algorithms

Metaheuristic algorithms are high-level strategies designed to navigate the search space of complex optimization problems efficiently. These algorithms do not guarantee an optimal

solution, but are effective in finding very good solutions within a reasonable time frame, especially for problems where traditional methods fail due to the curse of dimensionality or the presence of numerous local optima.

4.3.1 Simulated Annealing

Simulated Annealing (Simulated Annealing (SA)) presents itself as a stochastic technique designed to approximate the global optimum of a given objective function. This method finds its roots in the physical process of annealing in metallurgy, which involves controlled heating and cooling of a material to increase the size of its crystals and reduce their defects. The SA algorithm mimics this process by introducing a metaphorical "temperature" that gradually decreases according to a predetermined schedule. Initially, the algorithm is more inclined to accept solutions that are worse than the current solution, allowing it to explore the solution space widely and avoid becoming trapped in local optima prematurely. As the temperature decreases, the algorithm becomes more selective, focusing on the optimal solution.

4.3.1.1 Application to the TSP

In the context of the TSP, the SA algorithm starts with an arbitrary solution, typically a random tour among the cities. Subsequently, it iteratively refines this tour by making slight modifications, such as swapping two cities in the tour sequence. The decision to accept a new solution is probabilistic and depends on both the difference in solution quality and the current temperature. This method's ability to accept worse solutions decreases probabilistically over time as the temperature lowers, simulating the cooling process in annealing.

Table 4.3: Results of the Simulated Annealing algorithm

Instance	Time (ms)	Tour Length	Optimal Length	Gap
berlin52	28982	7544.37	7542.00	0.03
87eil76	39587	572.81	538.00	6.47
29lin105	52928	14993.92	14379.00	4.28
44d198	102362	16318.76	15780.00	3.41
49eil51	131542	434.25	426.00	1.94
pr124	207125	60675.98	59030.00	2.79
67u574	398181	43936.09	36905.00	19.05
lin318	506369	47651.44	42029.00	13.38
fl1577	1514830	27584.16	22249.00	23.98
rl5915	22563370	680777.61	565530.00	20.38

Pseudocode The pseudocode for applying SA to solve the TSP is outlined below. This algorithmic representation emphasizes the iterative improvement of a tour, taking into account the acceptance of suboptimal solutions based on a decreasing temperature schedule

to escape local minima.

Algorithm 7 Simulated Annealing

```
1: procedure SATSP(cities, initialTemp, coolingRate, stoppingTemp)
2:   bestSolution  $\leftarrow$  generateInitialSolution(cities)
3:   currentSolution  $\leftarrow$  bestSolution
4:   currentTemp  $\leftarrow$  initialTemp
5:   while currentTemp > stoppingTemp do
6:     newSolution  $\leftarrow$  perturbSolution(currentSolution, cities)
7:     currentCost  $\leftarrow$  calculateCost(currentSolution)
8:     newCost  $\leftarrow$  calculateCost(newSolution)
9:     acceptanceProb  $\leftarrow$  AcceptanceProbability(currentCost, newCost, currentTemp)
10:    if acceptanceProb > randomValue(0, 1) then
11:      currentSolution  $\leftarrow$  newSolution
12:      if newCost < calculateCost(bestSolution) then
13:        bestSolution  $\leftarrow$  newSolution
14:      end if
15:    end if
16:    currentTemp  $\leftarrow$  updateTemperature(currentTemp, coolingRate)
17:  end while
18:  return bestSolution
19: end procedure
```

In this pseudocode, *generateInitialSolution* creates a random initial tour of the cities. *perturbSolution* slightly alters the current solution, typically by swapping two cities in the tour. *calculateCost* evaluates the total distance (or cost) of a tour. *calculateAcceptanceProbability* determines the probability of accepting a new solution based on its cost, the cost of the current solution, and the current temperature, following the Metropolis criterion[0]. The temperature is updated in each iteration by the *updateTemperature* function, which reduces the temperature based on the cooling rate until reaching the stopping temperature.

4.3.2 Genetic Algorithms

Genetic Algorithms (GA) are a class of heuristic methods inspired by the principles of biological evolution and natural selection. Introduced by John Holland in the 1960s, these algorithms are particularly effective in tackling complex and nonlinear optimization problems, such as the TSP.

4.3.2.1 Fundamental Principles

Genetic algorithms operate on a population of potential solutions, encoded as "chromosomes". The fundamental principles include:

- **Selection:** The best (most "fit") solutions are more likely to be selected for reproduction.

- **Crossover:** Combination of parts from two parent solutions to create a new one.
- **Mutation:** Random modifications introduced in solutions to maintain genetic diversity.
- **Evolution:** New generations tend to improve over time, converging toward optimal or near-optimal solutions.

4.3.2.2 Application to the TSP

In the context of the TSP, a genetic algorithm typically operates as follows:

1. **Encoding:** Each tour is represented as a sequence of cities (chromosome).
2. **Initial population:** Generation of a random set of valid tours.
3. **Fitness function:** Evaluation of the length of each tour (fitness inversely proportional to length).
4. **Selection:** Choice of the best tours for reproduction (e.g., tournament or roulette selection).
5. **Crossover:** Combination of parts from two parent tours (e.g., partially mapped crossover - PMX).
6. **Mutation:** Small random modifications in tours (e.g., swapping two cities).
7. **Replacement:** Creation of a new generation by combining parents and offspring.
8. **Iteration:** Repetition of the process for a fixed number of generations or until reaching a stopping criterion.

Table 4.4: Results of the Genetic Algorithm

Instance	Time (ms)	Tour Length	Optimal Length	Gap
eil51	22815	440.85	426.00	3.49
berlin52	26104	7918.09	7542.00	4.99
eil76	44547	570.63	538.00	6.06
d198	108498	16548.42	15780.00	4.87
lin105	109871	14573.34	14379.00	1.35
pr124	137120	61453.95	59030.00	4.11
lin318	320190	45165.59	42029.00	7.46
u574	1260895	40516.22	36905.00	9.79
fl1577	4033113	24024.81	22249.00	7.98
rl5915	33367821	648205.16	565530.00	14.62

Algorithm 8 Genetic Algorithm for TSP

```
1: procedure GENETICALGORITHMTSP(cities, populationSize, generations, mutationRate)
2:   population  $\leftarrow$  InitializePopulation(cities, populationSize)
3:   for g  $\leftarrow$  1 to generations do
4:     EvaluateFitness(population)
5:     newPopulation  $\leftarrow$   $\emptyset$ 
6:     while  $|newPopulation| < populationSize$  do
7:       parent1  $\leftarrow$  SelectParent(population)
8:       parent2  $\leftarrow$  SelectParent(population)
9:       child  $\leftarrow$  Crossover(parent1, parent2)
10:      if Random() < mutationRate then
11:        Mutate(child)
12:      end if
13:      newPopulation  $\leftarrow$  newPopulation  $\cup$  {child}
14:    end while
15:    population  $\leftarrow$  newPopulation
16:  end for
17:  return BestSolution(population)
18: end procedure
```

4.3.2.3 Pseudocode**4.3.2.4 Advantages**

Genetic algorithms offer several significant advantages for solving the TSP:

- **Global exploration:** Ability to explore broad regions of the solution space, reducing the risk of premature convergence to local optima.
- **Parallelizability:** The population-based nature lends itself well to parallel computation, improving computational efficiency.
- **Flexibility:** Easily adaptable to variants of the TSP and other combinatorial optimization problems.
- **Robustness:** Ability to handle noisy or ill-defined objective functions.
- **Multiple solutions:** Can provide a set of good solutions, not just a single optimal solution.

4.3.2.5 Disadvantages

Despite their strengths, Genetic Algorithms also present some limitations:

- **Convergence time:** May require many generations to converge, especially for large-scale problems.
- **Parameter sensitivity:** Performance depends heavily on the choice of parameters such as population size, mutation rate, selection method, etc.

- **No guarantee of global optimum:** Like other metaheuristics, they do not guarantee finding the global optimal solution.
- **Encoding difficulty:** For some problems, effective representation of solutions as "chromosomes" can be complex.
- **Loss of diversity:** May suffer from premature convergence if not properly managed, leading to sub-optimal solutions.

4.3.2.6 Variants and Improvements

Numerous variants and improvements have been proposed to optimize the performance of Genetic Algorithms for the TSP:

- **Specialized crossover operators:** Such as Partially Mapped Crossover (PMX) (Partially Mapped Crossover) or Order Crossover (OX) (Order Crossover), specifically designed to preserve the relative order of cities.
- **Diversification techniques:** Introduction of mechanisms to maintain population diversity, such as "crowding" or "ecological niching".
- **Hybridization:** Combination with local search techniques (such as 2-opt) for fine optimization of solutions.
- **Parameter adaptation:** Strategies to dynamically adjust algorithm parameters during execution.
- **Alternative representations:** Use of different encodings for tours, such as path-based or adjacency-based representation.

Genetic Algorithms represent a powerful and flexible approach to tackling the TSP and other combinatorial optimization problems. Their ability to effectively explore vast and complex solution spaces makes them particularly suitable for large-scale instances or those with non-standard characteristics. However, the effectiveness of Genetic Algorithms depends heavily on the appropriate choice of genetic operators, parameters, and implementation strategies. Ongoing research in this field aims to further improve the performance and applicability of these algorithms to an increasingly wide range of optimization problems.

4.3.3 Tabu Search

Tabu Search (Tabu Search (TS)) is an optimization metaheuristic that extends local search by incorporating memory structures to guide the search process. Introduced by Fred Glover in 1986, this technique is particularly effective for combinatorial optimization problems such as the TSP.

4.3.3.1 Fundamental Principles

The key idea of TS is the use of adaptive memory (the tabu list) to avoid returning to recently visited solutions, thus allowing the algorithm to explore new areas of the solution space and potentially escape local optima. The tabu list stores characteristics of recent solutions or moves made, temporarily prohibiting their reuse.

4.3.3.2 Application to the TSP

In the context of the TSP, Tabu Search typically operates as follows:

1. Starts with an initial solution (a complete tour).
2. At each iteration, explores the neighborhood of the current solution (e.g., by swapping pairs of cities).
3. Selects the best non-tabu move, even if it worsens the current solution.
4. Updates the tabu list, adding the move just made and removing older moves if necessary.
5. Repeats the process for a predefined number of iterations or until a stopping criterion is met.

4.3.3.3 Pseudocode

4.3.3.4 Advantages

TS offers several significant advantages:

- **Avoids local optima:** The tabu list allows the algorithm to temporarily accept worsening moves, helping to escape local optima.
- **Efficient exploration:** Effectively balances exploration of new areas of the solution space and exploitation of found good solutions.
- **Adaptability:** Can be easily adapted to various combinatorial optimization problems.
- **Short and long-term memory:** Can incorporate short and long-term memory strategies to guide the search more intelligently.

4.3.3.5 Disadvantages

Despite its strengths, TS also presents some limitations:

- **Parameter sensitivity:** Performance can depend heavily on the choice of parameters, such as the tabu list size and aspiration criteria.

Algorithm 9 Tabu Search for TSP

```

1: procedure TABUSEARCHTSP(initialSolution, maxIterations, tabuListSize)
2:   currentSolution  $\leftarrow$  initialSolution
3:   bestSolution  $\leftarrow$  currentSolution
4:   tabuList  $\leftarrow$  initialize empty list
5:   for iteration  $\leftarrow$  1 to maxIterations do
6:     neighborhood  $\leftarrow$  generateNeighborhood(currentSolution)
7:     bestCandidate  $\leftarrow$  null
8:     for candidate in neighborhood do
9:       if (candidate better than bestCandidate AND move not in tabuList) OR
10:      (candidate better than bestSolution) then
11:        bestCandidate  $\leftarrow$  candidate
12:      end if
13:    end for
14:    currentSolution  $\leftarrow$  bestCandidate
15:    if currentSolution better than bestSolution then
16:      bestSolution  $\leftarrow$  currentSolution
17:    end if
18:    Update tabuList
19:  end for
20:  return bestSolution
21: end procedure

```

- **Computational complexity:** Exploring the neighborhood at each iteration can be computationally expensive for large-scale problems.
- **No guarantee of global optimum:** Like other metaheuristics, it does not guarantee finding the global optimal solution.
- **Implementation difficulty:** Efficient implementation of memory structures and aspiration criteria can be complex.

TS represents a powerful tool in the arsenal of metaheuristics for the TSP and other combinatorial optimization problems. Its ability to avoid local optima and effectively explore the solution space makes it particularly suitable for large-scale or complex problem instances. However, like many advanced techniques, its effectiveness depends on careful calibration and implementation.

4.4 Ant Colony Optimization

Ant Colony Optimization (ACO) is a metaheuristic algorithm inspired by the behavior of real ant colonies [0]. This approach simulates the way ants find the shortest paths between food sources and their nest, applying this concept to solving combinatorial optimization problems such as the TSP [0].

4.4.1 Fundamental Principles of ACO

ACO is based on several key principles:

1. **Pheromones:** Ants deposit a chemical substance called pheromone along the paths they follow. This serves as an indirect communication mechanism among ants [0].
2. **Evaporation:** Pheromones evaporate over time, allowing the algorithm to "forget" less optimal paths and prevent premature convergence [0].
3. **Choice probability:** Ants choose their path based on the amount of pheromones present and the distance between cities, with an element of randomness [0].

4.4.2 Application to the TSP

In the context of the TSP, ACO operates as follows:

1. **Initialization:** A small amount of pheromones is deposited on all possible paths.
2. **Tour Construction:** Each ant constructs a complete tour, choosing the next city to visit based on the formula:

$$P_{ij}^k = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum_{l \in N_i^k} (\tau_{il})^\alpha \cdot (\eta_{il})^\beta}$$

where:

- P_{ij}^k is the probability that ant k moves from city i to city j
 - τ_{ij} is the amount of pheromone on path (i, j)
 - η_{ij} is the inverse of the distance between cities i and j
 - α and β are parameters that control the relative importance of pheromones and distance
 - N_i^k is the set of cities not yet visited by ant k
3. **Pheromone Update:** After all ants have completed their tours, pheromones are updated according to the formula:

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}^k$$

where:

- ρ is the pheromone evaporation rate
- m is the number of ants

- $\Delta\tau_{ij}^k$ is the amount of pheromone deposited by ant k on path (i, j) , usually proportional to the quality of the tour found
4. **Iteration:** Steps 2 and 3 are repeated for a predefined number of iterations or until a termination criterion is reached.

The following pseudocode describes the basic implementation of ACO for the TSP:

```
1: procedure CONSTRUCTTOUR( $G, \alpha, \beta$ )
2:   Choose a random starting city
3:   while there are unvisited cities do
4:     Choose the next city using the probability formula
5:   end while
6:   return complete tour
7: end procedure
8: procedure UPDATEPHEROMONES( $G, \rho, Q$ )
9:   for each edge  $(i, j)$  in  $G$  do
10:     $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$ 
11:    for  $k = 1$  to  $m$  do
12:      if  $(i, j)$  is in  $tour_k$  then
13:         $\tau_{ij} \leftarrow \tau_{ij} + Q/length_k$ 
14:      end if
15:    end for
16:  end for
17: end procedure
```

The procedures ConstructTour and UpdatePheromones are represented in simplified form and can be further detailed in an actual implementation.

4.4.3 Variants and Improvements

Several variants of ACO have been proposed to improve algorithm performance on the TSP:

- **MAX-MIN Ant System (Max-Min Ant System (MMAS)):** Introduces upper and lower limits to the amount of pheromones, preventing stagnation [0].
- **Ant Colony System (ACS):** Uses a more aggressive transition rule and local pheromone update [0].
- **Rank-Based Ant System:** Modifies the pheromone update rule based on the quality of tours found [0].

4.4.4 Advantages and Challenges

ACO offers several advantages for solving the TSP:

Algorithm 10 ACO for the TSP

```
1: procedure ACO-TSP( $G, m, \alpha, \beta, \rho, Q, max\_iterations$ )
2:    $G \leftarrow$  graph of cities
3:    $m \leftarrow$  number of ants
4:    $\alpha, \beta \leftarrow$  control parameters
5:    $\rho \leftarrow$  pheromone evaporation rate
6:    $Q \leftarrow$  constant for pheromone deposit
7:    $max\_iterations \leftarrow$  maximum number of iterations
8:   Initialize pheromones  $\tau_{ij} = \tau_0$  for each edge  $(i, j)$  in  $G$ 
9:    $best\_tour \leftarrow \emptyset$ 
10:   $best\_length \leftarrow \infty$ 
11:  for  $iteration = 1$  to  $max\_iterations$  do
12:    for  $k = 1$  to  $m$  do
13:       $tour_k \leftarrow$  ConstructTour( $G, \alpha, \beta$ )
14:       $length_k \leftarrow$  CalculateTourLength( $tour_k$ )
15:      if  $length_k < best\_length$  then
16:         $best\_tour \leftarrow tour_k$ 
17:         $best\_length \leftarrow length_k$ 
18:      end if
19:    end for
20:    UpdatePheromones( $G, \rho, Q$ )
21:  end for
22:  return  $best\_tour, best\_length$ 
23: end procedure
```

- **Adaptability:** Can easily adapt to changes in the problem, such as adding or removing cities.
- **Solution quality:** Often finds high-quality solutions, especially for large-scale TSP instances.

However, ACO also presents some challenges:

- **Convergence:** May converge prematurely to sub-optimal solutions if parameters are not correctly calibrated.
- **Computation time:** For large-scale problems, may require significant computation time to reach high-quality solutions.
- **Parameterization:** Algorithm effectiveness depends heavily on parameter choice, which can be difficult to optimize.

4.4.5 Conclusions

ACO represents a powerful and flexible approach to tackling the Traveling Salesman Problem. Drawing inspiration from the natural foraging processes of ants, ACO offers a robust mechanism for exploring the solution space of the TSP, effectively balancing the

exploration of new solutions with the exploitation of accumulated information. Despite challenges related to parameterization and computation time, ACS continues to be an active area of research and development in the field of combinatorial optimization [0].

ANT COLONY SYSTEM BY DORIGO ET AL

5.1 Fundamentals

5.1.1 Biological Inspiration

The Ant Colony System (ACS) draws inspiration from real ants, which can efficiently find the shortest paths between food sources and their nest. This ability is attributed to the pheromone trails laid down by ants, facilitating an indirect form of communication that guides other ants towards optimal paths[0, 0].

Stigmergy, a mechanism of indirect coordination between agents, plays a crucial role in this process[0]. Ants deposit pheromones as they move, creating positive feedback that reinforces the most efficient paths over time[0].

5.1.2 Algorithmic Principles

The ACS leverages a colony of artificial ants that work cooperatively to explore and exploit solutions to optimization problems. The approach integrates both the exploration of new solutions and the exploitation of known good solutions through a balance between pheromone influence and heuristic desirability.[0, 0]

Dorigo et al. introduced several improvements over previous ant-based algorithms:

- A stronger state transition rule that better balances the exploration of new edges and the exploitation of accumulated knowledge about the problem.
- A global pheromone update rule applied only to edges belonging to the best tour.
- The application of a local pheromone update rule during solution construction.

In Figure 5.1

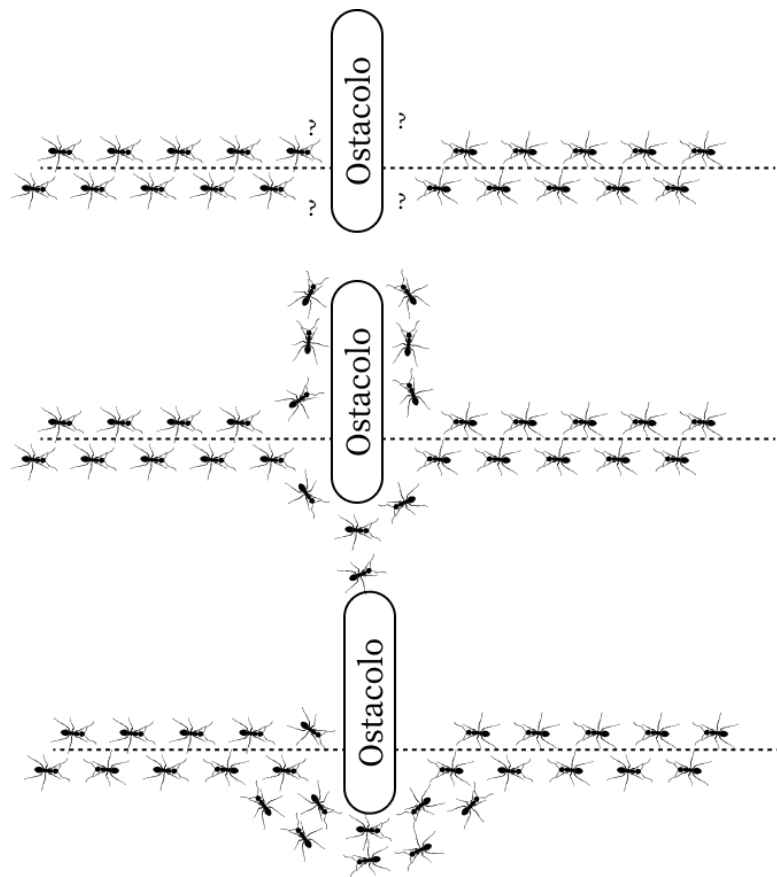


Figure 5.1: Ant behavior when facing an obstacle

1. Ants randomly choose among the available paths
2. Since ants move at an approximately constant speed, those that choose the lower, shorter path reach the opposite decision point faster than those that choose the upper, longer path.
3. Pheromone accumulates at a higher rate on the shorter path. Ants choose the lower path based on pheromone concentration.

5.1.3 Application to TSP

The TSP is particularly well-suited for the application of ACS due to its need for efficient path-finding solutions. The algorithm simulates ant behavior to construct paths, with the optimization process guided by both heuristic information (e.g., distances between cities) and pheromone trails that represent a form of learned experience[0, 0].

In the context of the TSP, each ant constructs a tour by visiting all cities exactly once. The choice of the next city to visit is influenced by both the distance (heuristic information) and the amount of pheromones on the edge connecting the cities (accumulated experience).

5.2 Implementation Details

5.2.1 Pheromone Update Rules

The pheromone update rules in ACS are crucial for guiding the search process towards promising areas of the solution space. These rules are divided into local and global updates.

5.2.1.1 Local Update Rule

Each time an ant traverses an edge (i, j) , it applies the local pheromone update rule to decrease the pheromone level, encouraging exploration by other ants. The local update rule is given by:

$$\tau_{ij} = (1 - \rho) \cdot \tau_{ij} + \rho \cdot \tau_0$$

where τ_{ij} is the pheromone concentration on edge (i, j) , ρ is the local evaporation rate ($0 < \rho < 1$) and τ_0 is the initial pheromone level.

The effect of this rule is to reduce the amount of pheromone on a visited edge, making that edge less desirable for subsequent ants. This increases the exploration of alternative paths and helps prevent premature stagnation on suboptimal solutions.

5.2.1.2 Global Update Rule

After all ants have completed their tours, the global update rule is applied to edges that belong to the best tour (either globally or in the current iteration). The global update enhances the pheromone trail on critical paths, making them more attractive for future ants. The global update rule is defined as:

$$\tau_{ij} = (1 - \alpha) \cdot \tau_{ij} + \alpha \cdot \Delta\tau_{ij}^{\text{best}}$$

where α is the global evaporation rate ($0 < \alpha < 1$) and $\Delta\tau_{ij}^{\text{best}}$ represents the pheromones deposited by the best ant, defined as:

$$\Delta\tau_{ij}^{\text{best}} = \begin{cases} (L_{\text{best}})^{-1} & \text{if } (i, j) \text{ belongs to the best tour} \\ 0 & \text{otherwise} \end{cases}$$

where L_{best} is the length of the best tour.

This rule intensifies pheromones on paths that have led to the best solutions, guiding the search towards promising regions of the solution space.

5.2.2 Ant Movement Rules

Ants select the next city to visit based on a probabilistic decision rule that balances the exploration of new paths with the exploitation of pheromone trails and heuristic information.

Table 5.1: Results of the ACS algorithm

Instance	Time (ms)	Tour Length	Optimal Length	Gap
eil51	17083	434.91	426.00	2.09
berlin52	18411	7606.66	7542.00	0.86
eil76	33136	554.04	538.00	2.98
d198	94445	16584.44	15780.00	5.10
lin105	123877	14981.78	14379.00	4.19
pr124	292446	60760.86	59030.00	2.93
lin318	338493	46646.52	42029.00	10.99
u574	452117	42512.54	36905.00	15.19
fl1577	2880286	25939.85	22249.00	16.59
rl5915	35872198	706188.19	565530.00	24.87

5.2.2.1 Transition Rule

In ACS the choice of the next city to visit is governed by the pseudo-random-proportional transition rule. Given an ant k in city i , the probability of moving to city j is determined by:

$$j = \begin{cases} 2 \max_{l \in N_i^k} \{[\tau_{il}] \cdot [\eta_{il}]^\beta\} & \text{if } q \leq q_0 \text{ (exploitation)} \\ J & \text{otherwise (balanced exploration)} \end{cases}$$

where q is a random number uniformly distributed in $[0, 1]$, q_0 is a parameter ($0 \leq q_0 \leq 1$) that determines the balance between exploitation and exploration, τ_{il} is the amount of pheromone on edge (i, l) , η_{il} is the heuristic value (usually the inverse of the distance), β is a parameter that controls the relative importance of heuristic information, and N_i^k is the set of cities not yet visited by ant k positioned in city i .

J is a random city chosen according to the probability distribution:

$$p_{ij}^k = \frac{[\tau_{ij}] \cdot [\eta_{ij}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}] \cdot [\eta_{il}]^\beta}$$

This transition rule favors the exploitation of accumulated information (when $q \leq q_0$) but leaves room for exploration (when $q > q_0$).

5.2.3 Pseudocode

Algorithm 11 ACS for the TSP

```

1: Initialize pheromone levels  $\tau_{ij} = \tau_0$  for all edges  $(i, j)$ 
2: Initialize the best global solution  $S_{gb}$  and its length  $L_{gb}$ 
3: for each iteration do
4:   for each ant  $k = 1, \dots, m$  do
5:     Place ant  $k$  on a random starting city
6:     Initialize partial tour  $S_k = \emptyset$ 
7:     repeat
8:       Select the next city  $j$  using the transition rule
9:       Add  $(i, j)$  to  $S_k$ 
10:      Apply local pheromone update to  $(i, j)$ 
11:       $i \leftarrow j$ 
12:    until tour  $S_k$  is complete
13:    if  $L(S_k) < L_{gb}$  then
14:       $S_{gb} \leftarrow S_k$ 
15:       $L_{gb} \leftarrow L(S_k)$ 
16:    end if
17:  end for
18:  Apply global pheromone update to the best tour  $S_{gb}$ 
19: end for
20: return the best solution found  $S_{gb}$ 

```

5.2.4 Complexity Analysis

The time complexity of ACS per iteration is $O(n^2m)$, where n is the number of cities and m is the number of ants. However, the number of iterations required for convergence can vary significantly depending on problem characteristics and algorithm parameters.

5.2.5 Key Parameters and their Impact

The effectiveness of ACS crucially depends on the proper setting of several parameters:

- ρ : Local pheromone evaporation rate
- α : Global pheromone evaporation rate
- β : Relative importance of heuristic information
- q_0 : Balance between exploitation and exploration
- m : Number of ants
- τ_0 : Initial pheromone level

The optimal choice of these parameters can vary depending on the specific problem and often requires a tuning phase.

5.3 Variants and Extensions

Since the original publication of ACS, numerous variants and extensions have been proposed:

- **MAX-MIN Ant System (MMAS):** Introduced by Stützle and Hoos, MMAS limits pheromone values within a predefined range to prevent premature stagnation. [0]
- **Rank-Based Ant System (Ant System with Rank-based Pheromone Update (ASrank)):** In this variant, ants are ranked according to the quality of their solutions, and the amount of pheromones deposited is proportional to their rank.
- **Hybrid ACS:** Combinations of ACS with other optimization techniques, such as genetic algorithms or Tabu Search, have shown promising results on various combinatorial optimization problems.

5.4 Applications beyond the TSP

Although initially developed for the TSP, ACS and its variants have found applications in a wide range of domains:

- Vehicle routing problems
- Scheduling and planning
- Frequency assignment in telecommunication networks
- Bioinformatics and protein folding
- Network optimization and grid computing
- Multi-objective optimization problems

The versatility of ACS makes it a powerful approach for tackling complex optimization problems in various fields of engineering and science.

RED-BLACK ANT COLONY SYSTEM

6.1 Introduction

This chapter presents a modified version of the ACS algorithm, called Red-Black Ant Colony System (RB-ACS), which aims to improve the performance of ACS on large-scale instances of the TSP. The RB-ACS incorporates several key improvements to the standard ACS approach, including parallel search, improved pheromone initialization, and differentiated parameter settings for the two ant groups. These modifications are designed to enable RB-ACS to explore the search space more effectively and efficiently converge towards optimal or near-optimal solutions, even for large-scale TSP problems [0].

6.2 The Red-Black Ant Colony System Algorithm (RB-ACS)

While the standard ACS algorithm has proven effective in solving TSP instances, its performance can deteriorate as problem size increases. The Red-Black Ant Colony System (RB-ACS) introduces several key modifications to the standard ACS approach to improve its scalability and solution quality for large-scale TSP problems [0].

6.2.1 Pheromone Initialization

In standard ACS, the initial pheromone level on all edges is set to a constant value τ_0 . In RB-ACS, the authors propose a different pheromone initialization scheme, where the initial pheromone level on each edge (r, s) is set according to the following equation:

$$\tau_{\text{init}}(r, s) = \frac{C}{\text{cost}(r, s)} \quad (6.1)$$

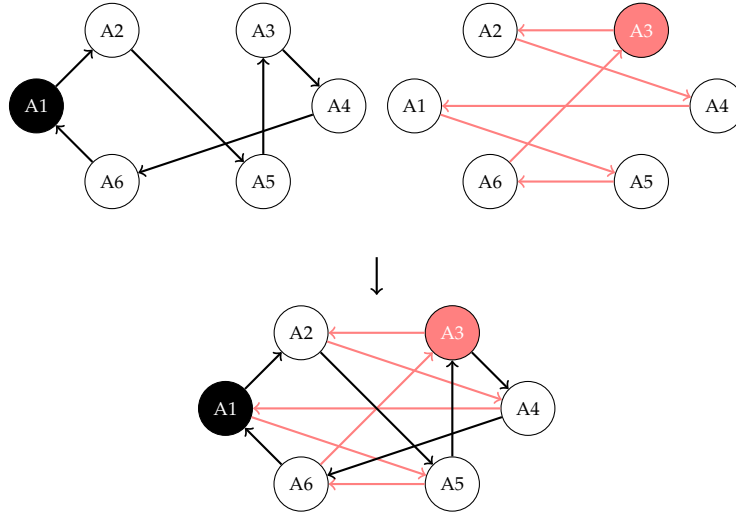


Figure 6.1: An example of tours constructed by red and black ants in the RB-ACS algorithm.

where C is a constant and $\text{cost}(r, s)$ is the distance between cities r and s . This modified pheromone initialization encourages ants to explore shorter edges, which are more desirable for the optimal tour, by assigning them higher initial pheromone levels [0].

6.2.2 Parallel Search with Red and Black Ant Groups

Another key modification in the RB-ACS algorithm is the use of two separate ant groups, called "red" and "black", which explore the search space in parallel. In standard ACS, a single group of ants constructs tours and ants can follow the paths of other ants, which can lead to premature convergence and getting stuck in local minima.

In RB-ACS, the red and black ant groups operate independently, each maintaining their own pheromone trails and searching for solutions without being influenced by the other group. This parallel search approach reduces the likelihood that the algorithm gets stuck in local minima, as the two ant groups can explore different regions of the search space simultaneously [0].

6.2.3 Differentiated Parameter Settings

In addition to the separate search process, the RB-ACS algorithm also employs different parameter settings for the red and black ant groups. Specifically, the local pheromone update rule and the pheromone evaporation rate can be set to different values for the two groups. This differentiation is inspired by the observed behavior of real ants, where different colonies or groups may exhibit distinct characteristics, such as pheromone deposition and evaporation rates [0].

By using separate parameter settings for red and black ants, the RB-ACS algorithm can further enhance the diversification of the search process, allowing the two groups to explore the solution space in a more complementary manner.

6.2.4 Improved Global Pheromone Update

In standard ACS, only the globally best ant is allowed to deposit pheromones during the global update phase. In RB-ACS, the authors propose a modified global update rule, where the two best ants from each of the red and black groups are allowed to update the pheromone levels. This parallel global update further strengthens the search towards high-quality solutions, as multiple promising paths are reinforced simultaneously [0].

6.2.5 Pseudocode

Algorithm 12 Red-Black Ant Colony System (RB-ACS) for the TSP

```

1: Initialize pheromone levels  $\tau_{ij} = \tau_{\text{init}}(i, j)$  for all edges  $(i, j)$ 
2: Initialize the best global solution  $S_{gb}$  and its length  $L_{gb}$ 
3: for each iteration do
4:   for each ant group  $g \in \{\text{red}, \text{black}\}$  do
5:     for each ant  $k = 1, \dots, m_g$  do
6:       Place ant  $k$  from group  $g$  on a random starting city
7:       Initialize partial tour  $S_k^g = \emptyset$ 
8:       repeat
9:         Select the next city  $j$  using the transition rule specific to group  $g$ 
10:        Add  $(i, j)$  to  $S_k^g$ 
11:        Apply local pheromone update to  $(i, j)$  according to group  $g$  rules
12:         $i \leftarrow j$ 
13:      until tour  $S_k^g$  is complete
14:      if  $L(S_k^g) < L_{gb}$  then
15:         $S_{gb} \leftarrow S_k^g$ 
16:         $L_{gb} \leftarrow L(S_k^g)$ 
17:      end if
18:    end for
19:  end for
20:  for each group  $g \in \{\text{red}, \text{black}\}$  do
21:    Select the two best ants from group  $g$ 
22:    Apply global pheromone update to the tours of these ants
23:  end for
24: end for
25: return the best solution found  $S_{gb}$ 

```

6.3 Conclusion

The RB-ACS algorithm presented in this chapter can be considered a promising approach for solving complex combinatorial optimization problems beyond the TSP, such as load balancing in telecommunication networks, economic load dispatch, process scheduling, and various other fields. The general principles of RB-ACS, including parallel search, differentiated parameter settings, and improved pheromone update strategies, can be

Table 6.1: Results of the RB-ACS algorithm

Instance	Time (ms)	Tour Length	Optimal Length	Gap
berlin52	34002	7681.45	7542.00	1.85
eil51	35268	445.59	426.00	4.60
eil76	40567	562.41	538.00	4.54
lin105	77720	14785.44	14379.00	2.83
d198	136998	17097.74	15780.00	8.35
pr124	468942	61167.76	59030.00	3.62
lin318	469358	46823.70	42029.00	11.41
u574	826684	43702.95	36905.00	18.42
fl1577	4210934	26788.92	22249.00	20.41
rl5915	21861319	716108.43	565530.00	26.63

potentially applied to a wide range of optimization problems, making it a valuable contribution in the field of swarm intelligence and metaheuristic algorithms.

EXPERIMENTAL RESULTS

In this chapter, an in-depth analysis of the experimental results obtained from the implementation and evaluation of different solution algorithms for the *Travelling Salesman Problem* (TSP) is presented. The experiments were conducted on a wide range of problem instances, varying in size and complexity, to assess the performance of the algorithms in terms of solution quality, execution time, and scalability.

7.1 Experimental Methodology

7.1.1 Setup

The experiments were executed on a computing cluster (DEI Blade) with nodes equipped with Intel Xeon Gold 5118 CPU @ 2.30/3.20GHz processors and 128 GB of RAM. All algorithms were implemented in Rust.

7.1.2 Problem Instances

A diversified set of TSP instances was used, primarily drawn from TSPLIB [0]. The instances range from small-sized problems (51 cities) to large-scale problems (up to 5915 cities). Some of the instances used include:

- eil51, berlin52, st70, eil76, pr76 (small-sized problems)
- lin105, pr124, lin318, rat575 (medium-sized problems)
- rat783, pr1002, pcb1173, d1655, fl1577, u2319 (large-sized problems)
- rl5915 (very large-scale problem)

7.1.3 Implemented Algorithms

The following algorithms were implemented and evaluated:

- Nearest Neighbor (NN) and NN with 2-Opt (NN2Opt)
- Simulated Annealing (SA) and SA with 2-Opt (SA2Opt)
- Genetic Algorithm (GA) and GA with 2-Opt (GA2Opt)
- Ant Colony System (ACS) and ACS with 2-Opt (ACS2Opt)
- Red-Black Ant Colony System (RBACS) and RBACS with 2-Opt (RBACS2Opt)

7.1.4 Evaluation Metrics

The main metrics used to evaluate algorithm performance are:

- **Execution time:** measured in milliseconds.
- **Solution quality:** length of the found tour.
- **Percentage gap:** calculated as $(length - optimum)/optimum * 100\%$, where "optimum" is the length of the known optimal tour for the instance.

7.2 Results Analysis

7.2.1 General Performance

From the analysis of the experimental data, some general trends emerge:

- The basic algorithms (NN, SA, GA, ACS, RB-ACS) show significantly improved performance when combined with the 2-Opt local optimization procedure.
- ACS and RBACS, together with their 2-Opt variants, tend to produce higher quality solutions compared to other approaches, especially for large-sized instances.
- GA shows highly variable performance, with some excellent results but also some extremely poor ones, especially on large-sized instances.

7.2.2 Detailed Analysis by Algorithm

7.2.2.1 Nearest Neighbor (NN and NN+2Opt)

NN is the fastest algorithm, with execution times ranging from a few milliseconds for small instances to a few seconds for very large instances. However, solution quality is generally lower compared to other approaches:

The addition of 2-Opt significantly improves solution quality, but increases computation time by an order of magnitude or more.

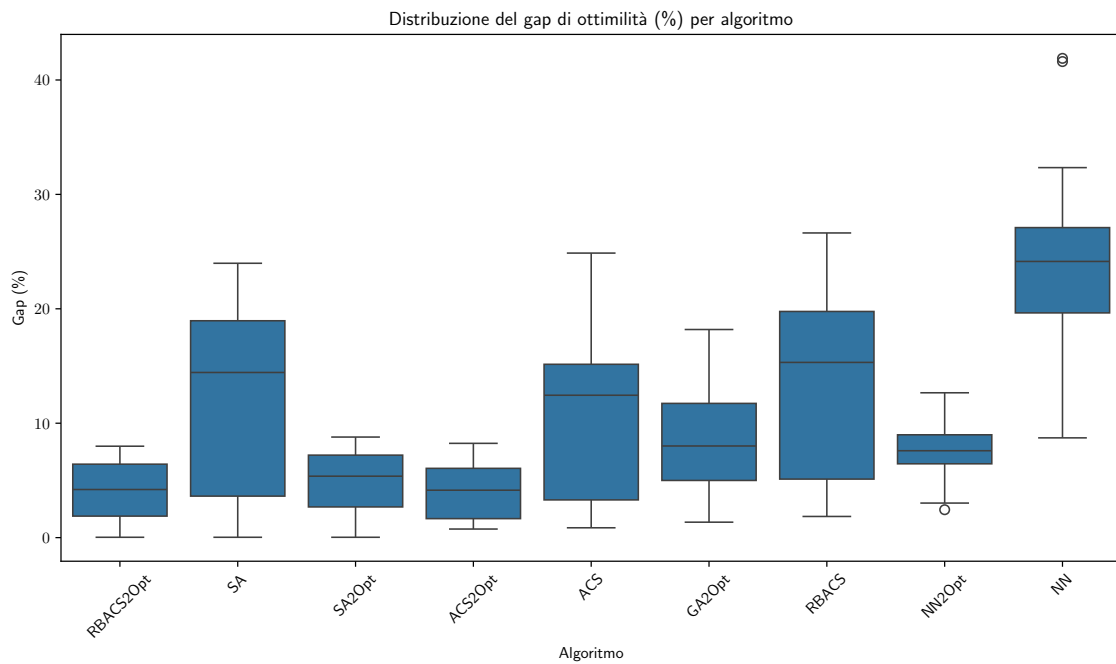


Figure 7.1: Distribution of optimality gap per algorithm

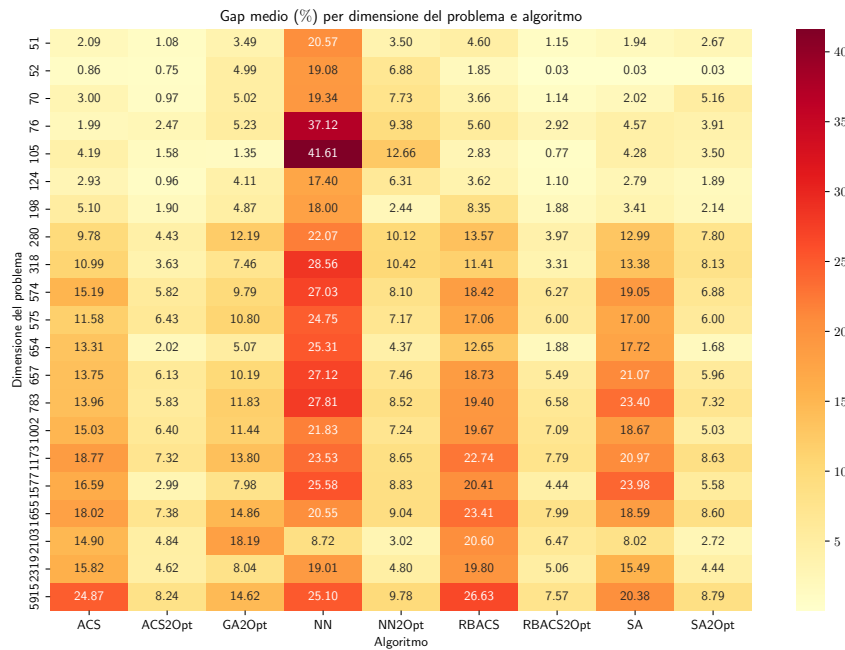


Figure 7.2: Heatmap of optimality gap

Table 7.1: Results NN vs NN+2Opt

	Instance	algorithm	Time (ms)	Tour Length	Optimal Length	Gap
0	berlin52	NN	14	8980.92	7542.00	19.08
1	berlin52	NN2Opt	124	8060.65	7542.00	6.88
2	d198	NN	183	18620.07	15780.00	18.00
3	d198	NN2Opt	4443	16165.31	15780.00	2.44
4	eil76	NN	17	711.99	538.00	32.34
5	eil76	NN2Opt	313	599.05	538.00	11.35
6	fl1577	NN	9547	27940.91	22249.00	25.58
7	fl1577	NN2Opt	200151	24214.30	22249.00	8.83
8	lin105	NN	43	20362.76	14379.00	41.61
9	lin105	NN2Opt	2959	16199.70	14379.00	12.66
10	lin318	NN	466	54033.58	42029.00	28.56
11	lin318	NN2Opt	9562	46408.41	42029.00	10.42
12	rl5915	NN	236473	707498.63	565530.00	25.10
13	rl5915	NN2Opt	6327036	620822.08	565530.00	9.78
14	u574	NN	1295	46881.87	36905.00	27.03
15	u574	NN2Opt	26923	39896.00	36905.00	8.10

7.2.2.2 Simulated Annealing (SA and SA2Opt)

SA offers a good compromise between solution quality and computation time:

Table 7.2: Results SA vs SA2Opt

	Instance	algorithm	Time (ms)	Tour Length	Optimal Length	Gap
0	berlin52	SA	28982	7544.37	7542.00	0.03
1	berlin52	SA2Opt	49481	7544.37	7542.00	0.03
2	d198	SA2Opt	93813	16118.48	15780.00	2.14
3	d198	SA	102362	16318.76	15780.00	3.41
4	eil76	SA	39587	572.81	538.00	6.47
5	eil76	SA2Opt	45721	569.29	538.00	5.82
6	fl1577	SA	1514830	27584.16	22249.00	23.98
7	fl1577	SA2Opt	2391057	23489.49	22249.00	5.58
8	lin105	SA	52928	14993.92	14379.00	4.28
9	lin105	SA2Opt	54366	14882.69	14379.00	3.50
10	lin318	SA2Opt	392433	45444.25	42029.00	8.13
11	lin318	SA	506369	47651.44	42029.00	13.38
12	rl5915	SA2Opt	18690779	615257.00	565530.00	8.79
13	rl5915	SA	22563370	680777.61	565530.00	20.38
14	u574	SA2Opt	369840	39443.68	36905.00	6.88
15	u574	SA	398181	43936.09	36905.00	19.05

SA2Opt generally produces better solutions than SA, but with significantly longer computation times.

7.2.2.3 Genetic Algorithm (GA and GA+2Opt)

GA shows the most variable performance among all tested algorithms:

Table 7.3: Results GA vs GA+2Opt

	Instance	algorithm	Time (ms)	Tour Length	Optimal Length	Gap
0	berlin52	GA2Opt	26104	7918.09	7542.00	4.99
1	berlin52	GA	29932	11009.32	7542.00	45.97
2	d198	GA2Opt	108498	16548.42	15780.00	4.87
3	d198	GA	137106	66868.76	15780.00	323.76
4	eil76	GA2Opt	44547	570.63	538.00	6.06
5	eil76	GA	45556	977.18	538.00	81.63
6	fl1577	GA	2264528	1116417.45	22249.00	4917.83
7	fl1577	GA2Opt	4033113	24024.81	22249.00	7.98
8	lin105	GA2Opt	109871	14573.34	14379.00	1.35
9	lin105	GA	164073	44653.50	14379.00	210.55
10	lin318	GA2Opt	320190	45165.59	42029.00	7.46
11	lin318	GA	429314	363904.18	42029.00	765.84
12	rl5915	GA	20712540	38904633.04	565530.00	6779.32
13	rl5915	GA2Opt	33367821	648205.16	565530.00	14.62
14	u574	GA	471427	457563.15	36905.00	1139.84
15	u574	GA2Opt	1260895	40516.22	36905.00	9.79

7.2.2.4 Ant Colony System (ACS and ACS+2Opt)

ACS and ACS+2Opt show very good performance across the entire range of instances:

Table 7.4: Results ACS vs ACS+2Opt

	Instance	algorithm	Time (ms)	Tour Length	Optimal Length	Gap
0	berlin52	ACS	18411	7606.66	7542.00	0.86
1	berlin52	ACS2Opt	20289	7598.44	7542.00	0.75
2	d198	ACS	94445	16584.44	15780.00	5.10
3	d198	ACS2Opt	101484	16079.65	15780.00	1.90
4	eil76	ACS2Opt	27703	558.76	538.00	3.86
5	eil76	ACS	33136	554.04	538.00	2.98
6	fl1577	ACS	2880286	25939.85	22249.00	16.59
7	fl1577	ACS2Opt	3852624	22914.07	22249.00	2.99
8	lin105	ACS2Opt	80683	14605.88	14379.00	1.58
9	lin105	ACS	123877	14981.78	14379.00	4.19
10	lin318	ACS2Opt	202670	43556.33	42029.00	3.63
11	lin318	ACS	338493	46646.52	42029.00	10.99
12	rl5915	ACS2Opt	34991642	612104.64	565530.00	8.24
13	rl5915	ACS	35872198	706188.19	565530.00	24.87
14	u574	ACS	452117	42512.54	36905.00	15.19
15	u574	ACS2Opt	490206	39051.51	36905.00	5.82

ACS2Opt tends to produce slightly higher quality solutions compared to ACS, but with longer computation times.

7.2.2.5 Red-Black Ant Colony System (RBACS and RBACS2Opt)

RBACS and RBACS2Opt show competitive and sometimes superior performance compared to ACS and ACS2Opt:

Table 7.5: Results RB-ACS vs RB-ACS+2Opt

	Instance	algorithm	Time (ms)	Tour Length	Optimal Length	Gap
0	berlin52	RBACS	34002	7681.45	7542.00	1.85
1	berlin52	RBACS2Opt	34159	7544.37	7542.00	0.03
2	d198	RBACS	136998	17097.74	15780.00	8.35
3	d198	RBACS2Opt	283417	16076.46	15780.00	1.88
4	eil76	RBACS	40567	562.41	538.00	4.54
5	eil76	RBACS2Opt	40672	557.14	538.00	3.56
6	fl1577	RBACS	4210934	26788.92	22249.00	20.41
7	fl1577	RBACS2Opt	6887456	23236.46	22249.00	4.44
8	lin105	RBACS	77720	14785.44	14379.00	2.83
9	lin105	RBACS2Opt	262217	14489.08	14379.00	0.77
10	lin318	RBACS	469358	46823.70	42029.00	11.41
11	lin318	RBACS2Opt	603140	43421.71	42029.00	3.31
12	rl5915	RBACS	21861319	716108.43	565530.00	26.63
13	rl5915	RBACS2Opt	28403435	608329.94	565530.00	7.57
14	u574	RBACS	826684	43702.95	36905.00	18.42
15	u574	RBACS2Opt	899785	39219.59	36905.00	6.27

RB-ACS2+Opt appears particularly effective on large-sized instances, offering a good compromise between solution quality and computation time.

7.2.3 Scalability Analysis

To evaluate the scalability of the algorithms, we analyzed how execution time and solution quality vary as problem size increases:

- NN and NN2Opt show the best scalability in terms of execution time, but solution quality degrades rapidly for larger instances.
- SA and SA2Opt maintain good solution quality even for large instances, but computation time grows rapidly.
- GA and GA2Opt show poor scalability both in terms of time and solution quality for very large instances.
- ACS, ACS2Opt, RBACS and RBACS2Opt offer the best compromise between execution time scalability and maintaining solution quality for large-sized instances.

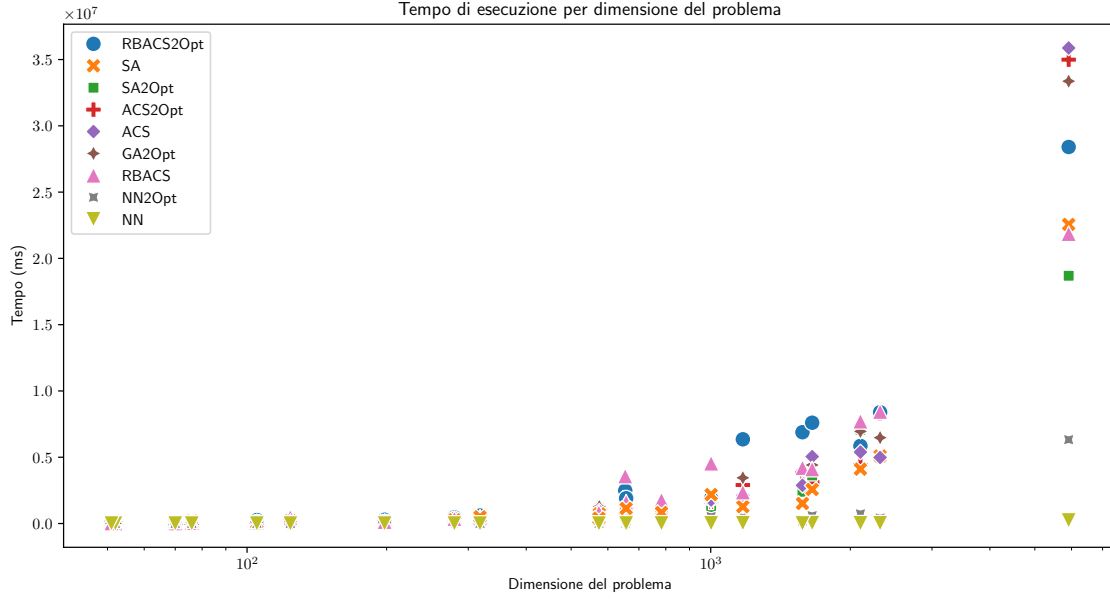


Figure 7.3: Execution time by problem size

7.3 Discussion

The experimental results highlight several key points:

1. The importance of local optimization: the addition of 2-Opt consistently improves the performance of all tested algorithms.
2. The superiority of ant colony-based approaches (ACS and RBACS) for medium and large-sized instances.
3. The variability of genetic algorithm performance, which suggests the need for careful parameter calibration to obtain good results.
4. The trade-off between computation time and solution quality, with more sophisticated algorithms requiring more time but producing better solutions.

RBACS and RBACS2Opt emerge as promising approaches, offering competitive and sometimes superior performance compared to ACS and ACS2Opt, especially on large-sized instances. However, the choice of the optimal algorithm depends on the specific requirements of the application, balancing the need for high-quality solutions with computation time constraints.

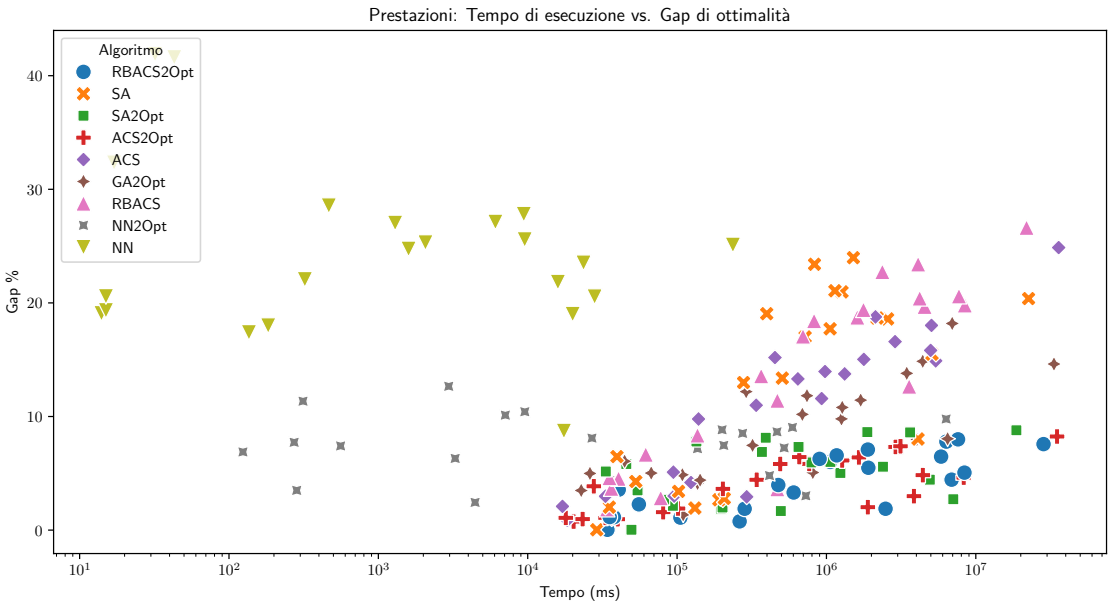


Figure 7.4: Gap by problem size

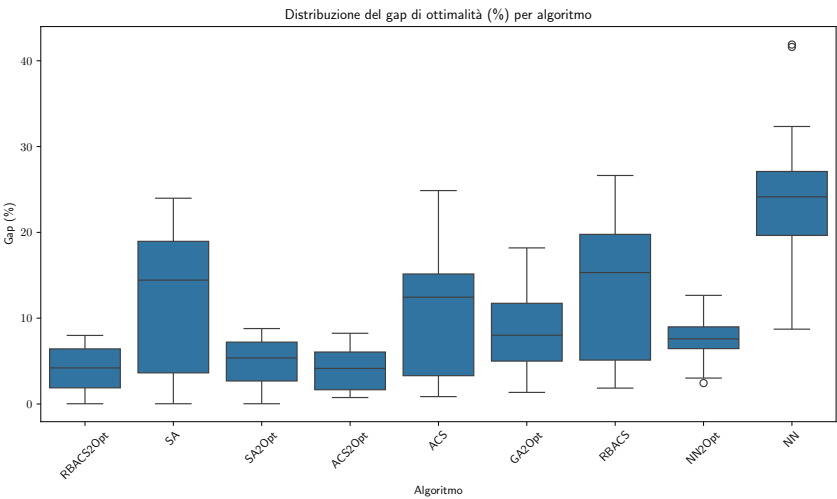


Figure 7.5: Comparison of top 3 algorithms: Gap

CONCLUSIONS

In conclusion, this thesis has provided an in-depth and comparative analysis of different algorithmic approaches for solving the Travelling Salesman Problem, with particular attention to performance on large-sized instances. The results obtained not only confirm the effectiveness of well-established metaheuristic approaches such as Ant Colony System, but also introduce a variant, the Red-Black Ant Colony System, which shows potential for further improvements and applications.

8.1 Final Reflections

The evolution of algorithms for the TSP reflects a broader trend in the field of combinatorial optimization: the search for a balance between solution quality and computational efficiency. This thesis has demonstrated that:

- There is no universally "best algorithm" for the TSP. The choice of the optimal approach depends on the specific characteristics of the problem, the instance size, and computational constraints.
- The hybridization of techniques, such as integrating local search procedures (2-Opt) into population-based metaheuristics, can lead to significant performance improvements.
- Scalability remains a crucial challenge, especially for very large-scale problems (over 10,000 cities). In this context, approaches like RBACS that offer greater parallelization opportunities become particularly relevant.
- Algorithm robustness, that is their ability to maintain good performance across a wide range of different instances, is a critical factor for practical applications.

8.2 Concluding Remarks

The Travelling Salesman Problem, despite its apparent simplicity, continues to challenge researchers and practitioners, pushing the boundaries of combinatorial optimization and theoretical computer science.

BIBLIOGRAPHY

- [0] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman & Co., 1979.
- [0] D. E. Knuth. "Big Omicron and Big Omega and Big Theta". In: *ACM SIGACT News* 8.2 (1976), pp. 18–24.
- [0] M. Dorigo and L. M. Gambardella. "Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem". In: *IEEE Transactions on Evolutionary Computation*. Vol. 1. 1. 1997, pp. 53–66.
- [0] M. Held and R. M. Karp. "A Dynamic Programming Approach to Sequencing Problems". In: *Journal of the Society for Industrial and Applied Mathematics* 10.1 (1962), pp. 196–210.
- [0] C.M. Institute. *P vs NP Problem*. 2000. URL: <http://www.claymath.org/millennium-problems/p-vs-np-problem>.
- [0] R. M. Karp. "Reducibility Among Combinatorial Problems". In: *Complexity of Computer Computations* (1972), pp. 85–103.
- [0] D. L. Applegate et al. *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ, USA: Princeton University Press, 2007.
- [0] G. Reinelt. *TSPLIB - A Traveling Salesman Problem Library*. Available online at <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/>. 1991. URL: <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/>.
- [0] W. J. Cook. "In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation". In: *Princeton University Press* (2011).
- [0] D. S. Johnson and L. A. McGeoch. "The Traveling Salesman Problem: A Case Study in Local Optimization". In: *Local Search in Combinatorial Optimization* (2002). Ed. by E. H. Aarts and J. K. Lenstra, pp. 215–310.
- [0] T. H. Cormen et al. *Introduction to Algorithms*. 3rd. Cambridge, MA: MIT Press, 2009. ISBN: 9780262033848.
- [0] C. H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Mineola, NY: Dover Publications, 1998. ISBN: 9780486402581.

- [0] E. L. Lawler et al. "The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization". In: *Wiley-Interscience Series in Discrete Mathematics and Optimization* (1985).
- [0] E. H. Aarts and J. H. Korst. *Simulated Annealing and Boltzmann Machines: A Stochastic Approach to Combinatorial Optimization and Neural Computing*. Chichester: John Wiley & Sons, 1989. ISBN: 9780471921462.
- [0] G. Gutin and A. P. Punnen. "Heuristic Algorithms for the Traveling Salesman Problem". In: *The Traveling Salesman Problem and Its Variations*. Boston, MA: Springer, 2016, pp. 223–269. ISBN: 9781461302374.
- [0] S. Lin and B. W. Kernighan. "An effective heuristic algorithm for the traveling salesman problem". In: *Oper. Res.* 21 (1973), pp. 498–516.
- [0] N. Metropolis et al. "Equation of state calculations by fast computing machines". In: *The Journal of Chemical Physics* 21.6 (1953), pp. 1087–1092.
- [0] M. Dorigo, V. Maniezzo, and A. Coloni. "Ant system: optimization by a colony of cooperating agents". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 26.1 (1996), pp. 29–41.
- [0] M. Dorigo and L. M. Gambardella. "Ant colony system: a cooperative learning approach to the traveling salesman problem". In: *IEEE Transactions on evolutionary computation* 1.1 (1997), pp. 53–66.
- [0] M. Dorigo, M. Birattari, and T. Stützle. "Ant colony optimization". In: *IEEE computational intelligence magazine* 1.4 (2006), pp. 28–39.
- [0] T. Stützle and H. H. Hoos. "MAX-MIN ant system". In: *Future generation computer systems* 16.8 (2000), pp. 889–914.
- [0] M. Dorigo, G. Di Caro, and L. M. Gambardella. "Ant colony optimization: a new meta-heuristic". In: *Proceedings of the 1999 congress on evolutionary computation-CEC99 (Cat. No. 99TH8406)*. Vol. 2. IEEE, 2004, pp. 1470–1477.
- [0] B. Bullnheimer, R. F. Hartl, and C. Strauss. "An improved ant system algorithm for the vehicle routing problem". In: *Annals of operations research* 89 (1999), pp. 319–328.
- [0] M. Dorigo and T. Stützle. "Ant colony optimization: A literature review". In: *Swarm Intelligence* 4 (2010), pp. 1–41.
- [0] M. Dorigo and L. M. Gambardella. "Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem". In: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), pp. 53–66.
- [0] E. Bonabeau, M. Dorigo, and G. Theraulaz. "Stigmergy: A novel mechanism of communication and control". In: *MIT Press* (1999).
- [0] J.-L. Deneubourg et al. "The Self-Organizing Exploratory Pattern of the Argentine Ant". In: *Journal of Insect Behavior* 3.2 (1990), pp. 159–168.

- [0] L. M. Gambardella and M. Dorigo. "Ant-Q: A Reinforcement Learning approach to the traveling salesman problem". In: *International Conference on Machine Learning*. 1995, pp. 252–260.
- [0] T. Stuetzle and H. H. Hoos. "Max-Min Ant System for the Traveling Salesman Problem". In: *Proceedings of the IEEE International Conference on Evolutionary Computation*. 1997, pp. 309–314.
- [0] M. H. Md. Rakib Hassan Md. Kamrul Hasan. "An Improved ACS Algorithm for the Solutions of Larger TSP Problems". In: *Proceedings of the ICEECE*. Dhaka, Bangladesh, 2013, pp. 1–5.

